claude-windows / 7fe4ef45-d30a-448e-98eb-4da4fba244f1

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fe4ef45-d30a-448e-98eb-4da4fba244f1\subagents\workflows\wf_b531eaef-39c\agent-a9b14109bd9d43fa8.jsonl`
- SHA-256: `3d8deab1e6c6713153484afbe22caaab43ad905a142c02cf1d19dba776384678`
- Source modified: `2026-06-19T17:03:03+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_b531eaef-39c`
- session_id: `7fe4ef45-d30a-448e-98eb-4da4fba244f1`

Transcript

1. user (2026-06-19T16:59:09.582Z)

You are an adversarial design reviewer. Be skeptical; the goal is to find real flaws BEFORE we build.

CONTEXT ? badminton/racket-sports rally detector. Pipeline: WASB shuttle DETECTOR (heatmap ? per-frame
candidate list) ? single-target online TRACKER (collapses to ONE (x,y) per frame) ? velocity windower ? rallies.

MEASURED (GX010142, owner's anchor clip, on the 21-25s window, from the cached det_raw.jsonl detector candidates):
- WASB's detector emits >=2 candidates on 31% of detected frames (cached, never used past the tracker).
- During the 21-25s rally: 93% of frames carry the ADJACENT-court shuttle (x-norm ~0.88, what the tracker followed),
  83% ALSO carry a PROMINENT-court shuttle (x-norm ~0.56), 76% carry BOTH at once.
- A greedy nearest-neighbour reconstruction of the prominent-court candidates yields a CONTINUOUS rally arc
  (75% window coverage, 431px vertical excursion, 8.6px median per-frame jump, ~2.4s sustained) ? VISUALLY CONFIRMED
  as a real foreground-court rally (player mid-stroke ? net ? second player), which the single-target tracker discarded.

PROPOSED DESIGN ("the Which problem"):
Split the tracker's two conflated jobs:
 - Layer 1 "see everything": a MULTI-TARGET linker over the ALREADY-CACHED det_raw.jsonl candidates ? a SET of shuttle
   tracks per region (top-K), each with trajectory + duration + motion-energy. Pure-Python over the cache, ZERO GPU,
   WASB stays frozen (MIT).
 - Layer 2 "pick the match's story": score each track by court-membership (the existing prominent-court polygon +
   point_in_polygon) + rally-likeness (motion energy, vertical excursion, >=2 net-crossings/Gate-A, ends on
   shuttle-floor-contact) and SELECT the track(s) whose story is the match's; feed those to the windower.
Claimed benefits: recall-POSITIVE (recovers the discarded prominent rally ? a G1 miss) AND precision-positive (drops the
adjacent rally) ? unlike today's court_gate which is a subtractive post-filter on the single track (precision-only). Also
claimed to SUBSUME the "ghost" (a stuck-pixel false-lock track has ~0 motion energy ? never selected). Lands default-OFF,
flag-gated, measured on cached candidates + golden labels with no GPU; single-track tracker stays the default.

YOUR LENS:
Attack the RECALL/PRECISION claim. The design says selecting "any sustained track in the prominent court" is recall-positive AND precision-positive. Where does this MANUFACTURE false rallies (e.g. selecting an adjacent-court track that briefly enters the polygon, or selecting BOTH tracks and double-counting, or a warm-up/practice rally on the prominent court)? Is "recall-positive" actually true or could net F1 drop? Give concrete failure modes + mitigations.

Return concrete, technically-grounded risks with severity + mitigations. Mark kills_design=true ONLY for a fundamental unmitigable flaw.

2. assistant (2026-06-19T16:59:12.310Z)

I'll investigate the actual code and cached data to ground my adversarial review in reality rather than just reasoning from the design description.

3. user (2026-06-19T16:59:15.684Z)

Your configuration specifies to merge with the ref 'refs/heads/docs/court-floor-rally-end' from the remote, but no such ref was fetched.

4. user (2026-06-19T16:59:15.861Z)

docs\GOLDEN_SET_IMPLEMENTATION.md
docs\GOLDEN_SET_PHASE1_VIDEOS.md
docs\PMF_MARKET_RESEARCH.md
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md
docs\HOSTING_PLAN.md
docs\UI_PROPOSAL.md
docs\UI_ALPHA_EXECUTION_PLAN.md
docs\FIELD_VISIT_ANSWER_SHEET.md
docs\FIELD_VISIT_PLAYBOOK.md
docs\I18N_PLAN.md
docs\PMF_FIELD_SIGNALS.md
docs\PRODUCT_FIELD_ASSOCIATE_FLYER.md
docs\VIDEO_LOCALITY_MODEL.md
docs\GOLDEN_SET_VENDORING.md
docs\STORAGE_SHARING_MODEL.md
docs\PER_VIDEO_OUTPUT_LAYOUT.md
docs\ABLATION_LAYER.md
docs\ANALYZERS_AND_RECONCILERS.md
docs\CODE_AUDIT_AND_TEST_HARDENING.md
docs\COMMERCIALIZATION.md
docs\DEFERRED_HARDENING_PLAN.md
docs\EVALUATION.md
docs\EXECUTION_POLICY.md
docs\EXPERIMENT_HARNESS.md
docs\HARDENING_LOOP_HANDOFF.md
docs\PERSONALIZATION_PLAN.md
docs\PLATFORM_ARCHITECTURE.md
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
docs\ROADMAP.md
docs\ROORA_LABELING_GUIDELINES.md
docs\TRACKNET_WSL_SETUP.md
docs\KHELSUTRA_PRODUCT_OVERVIEW.md
docs\PROJECT_DOC_TEMPLATE.md
docs\GOLDEN_DATA_SHARING.md
docs\GOLDEN_REGRESSION_FIXTURES.md
docs\REAL_FOOTAGE_VALIDATION_REPORT.md
docs\R1_MILESTONE_LAUNCH_REPORT.md
docs\VIDEO_RECORDING_GUIDELINES.md
docs\DESKTOP_APP_PLAN.md
docs\BACKLOG.md
docs\CODE_MAP.md
docs\GOLDEN_VIDEOS.md

```
docs\HOW_RALLY_DETECTION_WORKS.md
docs\MULTI_SIGNAL_FUSION_PLAN.md
docs\NEXT_STEPS.md
docs\QUALITY_ITERATIONS.md
docs\R2_EVAL_METRIC_DESIGN.md
docs\RALLY_DETECTION_GAP_CLOSURE.md
docs\RALLY_DETECTION_QUALITY_REPORT.md
docs\RALLY_QUALITY_RESEARCH.md
docs\README.md
docs\PROMINENT_COURT_DETECTION.md
```

5. assistant (2026-06-19T16:59:18.018Z)

Let me look at the cached data and key code/docs to ground the review.

6. user (2026-06-19T16:59:19.459Z)

```
1       # Prominent-Court Detection ? kill background-court false positives (multicourt)
2
3       > **Status:** `DRAFT` ? design + experiment **owner-gated for any default flip** ·
**Owner:** Avi · **Created:** 2026-06-19 · **Last updated:** 2026-06-19
4       > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5       > **Tracking anchors:** §8 progress tracker is the source of truth; pointer in memory
`current-state`.
6       > **Relation to existing docs:** attacks gap **G2** ("precision ? local over-fires,
worst on multicourt") from [`RALLY_DETECTION_GAP_CLOSURE.md`](RALLY_DETECTION_GAP_CLOSURE.md);
it is the **shuttle-side** counterpart of the person-side **Gate B / court mask** in
[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md). Complements the held-shuttle
**`shuttle_player_colocation`** (G3) ? court-localization (G2) and held-shuttle (G3) are
independent precision levers.
7       > **Honesty note:** claims are tagged `[verified]` (checked vs code/measured) /
`[design]` (proposed). Every signal lands **flag-gated, default-OFF**, promoted only on measured
lift per [`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md) + [[decision-rigor-norm]].
8
9       ---
10
11      ## 0. TL;DR
12      In **multicourt** footage the camera sees several courts. The shuttle detector (WASB)
tracks *any* shuttle in frame, so a rally on a **background / adjacent court** becomes a
predicted rally on the **main match** ? a false positive. The fix: identify the **prominent
court** (the foreground court that occupies the majority of the frame, where the match being
filmed is played) and **keep only rallies whose shuttle lives inside it**. Mechanically this re-
defines the shuttle signal from `(x, y, visible)` to `(x, y, visible-AND-inside-the-prominent-
court)` *before* windowing ? so a background-court shuttle reads as "not there" and never forms
a rally. A flat 2D floor polygon is the MVP; a **pseudo-3D play-volume** (court floor + the air
column above it) is the refinement so a high clear over the prominent court isn't wrongly
rejected. Lands as a **default-OFF experiment**; the flip is gated on a Launch Report ([[launch-
report-convention]]).
13
14      ## 1. Problem & motivating example `[verified]`
15      - **Measured gap (G2):** local over-fires ~2× vs Gemini, and the over-fire is **worst on
multicourt** ? even among long (>7s) rallies, multicourt over-fires **1.58×** (background-court
games are *real* long rallies, just on the wrong court). See `RALLY_DETECTION_GAP_CLOSURE.md` §3
+ the P7 long-rally lens.
16      - **Concrete anchor ? GX010142 rally #1 (2026-06-19).** Owner observation: *"the first
rally is just a shuttle flying in the non-prominent court? while the prominent-court players
have not started."* Confirmed from the cached WASB trajectory
(`output/wasb/GX010142_1080p__*_wasb.csv`): rally #1 (`0.72?3.65s`, 2.94s) has a shuttle
centroid at **meanX ? 1616** on a 1920-wide frame (?84% to the right) ? spatially apart from the
central cluster where most rallies live (meanX ? 1000?1300). Several other suspicious rallies
(#3, #9, #12, #16) cluster in the same far-right band ? consistent with a **right-side adjacent
court**.
17      - **Why a 1D threshold is not enough `[verified]`:** the courts do not separate cleanly
on X or Y alone (rally Y-centroids overlap 250?620 px across courts; the far-right band mixes
```

with foreground play). The courts are **2D regions** under perspective ? so we need the court *geometry*, not a coordinate cutoff. This is the owner's "pseudo-3D / court model" intuition, validated by the data.

2. Goal & non-goals

- **Goal:** on multicourt clips, drop rallies whose shuttle is outside the prominent court, **without losing real prominent-court rallies** ? measured per-video, no regression on single-court clips (where it must be a no-op or a do-no-harm).
- **Non-goals (v1):** multi-camera; tracking players across courts; full court-line homography for scoring; anything biometric. Single-court footage is explicitly a **no-op** (one court ? everything is "prominent").

3. Decisions locked

#	Decision	Rationale
D1	The prominent court is a **2D region (polygon) in image space**, refined to a **pseudo-3D play volume**	1D thresholds don't separate courts under perspective (§1)
D2	Gate the **shuttle trajectory** by the region **before windowing** (re-define visibility)	Stops a background rally at the source ? no window ever forms there. Reuses `point_in_polygon`
D3	**Default-OFF, flag-gated; single-court = no-op**	Zero-regression rollout ([[weak-signals-retained]]); no behaviour change unless a polygon is supplied + the flag set
D4	Reuse the existing **`court_polygon`** config + `point_in_polygon`; do NOT invent a new geometry stack	`FusionConfig.court_polygon` + `fusion_features.point_in_polygon` already exist `[verified]`
D5	The prominent court can be **supplied (config) OR auto-derived**; auto-derivation is its own measured step	Config polygon is the cheapest first experiment; auto is the product path

4. Design

4.1 The signal change (the core experiment)

Today the windower consumes the raw WASB trajectory: a per-frame `(Frame, Visibility, X, Y)`. The change is a **pre-windowing filter**:

```
for each frame f:  if shuttle_visible(f) and NOT inside_prominent_court(X_f, Y_f):  mark f as NOT visible
```

A background-court shuttle is then indistinguishable from "no shuttle" ? the velocity windower never opens a rally there. This is purely **subtractive** on the shuttle stream (it can only remove visibility, never add it), so on a correctly-drawn prominent court it cannot manufacture rallies ? only suppress out-of-court ones. (Mirror of `rally_gate`'s "do-no-harm" discipline.)

4.2 Identifying the prominent court (four routes, cheapest first)

1. **Configured polygon** (MVP, `[design]?buildable now`): a normalized `[[x,y],?]` court polygon per camera/clip, exactly the existing `FusionConfig.court_polygon`. Draw it once from the dense shuttle/player cluster. Good enough to validate the whole concept on GX010142 + the multicourt golden clips.
2. **Player-location auto-derivation** (`[design]`): run `PersonDetector.detections_per_frame` `[verified] exists`, take the **largest, most-central, most-persistent** player cluster (the foreground players are bigger boxes lower in frame), and fit the prominent-court polygon to where *those* players' feet live. "Prominent = where the match's players are." This is why the owner asked to **extract person features** ? players anchor the court.
3. **Shuttle-density auto-derivation** (`[design]`): the prominent court is the densest sustained shuttle-activity region; cluster the visible trajectory and take the dominant mode. Person-free fallback.
4. **Court-line detection** (`[design]`, heaviest): Hough/segmentation court lines ? the largest court quad. Most accurate, most work; deferred.

v1 ships route 1 (config) + measures; route 2 (player-anchored) is the product upgrade.

4.3 Pseudo-3D play volume (the refinement)

A flat floor polygon rejects a shuttle at the top of a high clear *over the prominent court* (it projects above the floor quad). The pseudo-3D model treats the prominent court as a **floor quad + a vertical air column**: accept a shuttle if its image point lies within the floor polygon **expanded upward** by a height envelope (the max plausible shuttle height projected through the camera). Practically: the floor quad plus an upward `pad` (perspective-aware ? larger near the net/far baseline). Start with a generous fixed upward pad (cheap, MVP); upgrade to a per-camera height envelope only if measurement shows real rallies being clipped. This keeps the floor tight (rejects adjacent-court floor play) while not clipping legitimate high shuttles.

4.4 Reuse map `[verified]`

Need	Existing code
point-in-region test	`backend/pipeline/detectors/fusion_features.py::point_in_polygon`
normalized court polygon config	`backend/config/models.py::FusionConfig.court_polygon` (today: person/Gate-B only)
player boxes per frame	`backend/pipeline/detectors/person_detector.py::PersonDetector.detections_per_frame`
trajectory parse + windowing	`backend/pipeline/detectors/tracknet_runner.parse_trajectory_csv`, `backend/eval/windowing.py`
per-video scoring	`backend/eval/rally_seg_eval.py` (+ the `--stratify` multicourt split, #220)

New code is small: a `prominent_court_gate(points, polygon, height_pad)` that filters trajectory points, threaded into the windowing input behind a default-OFF flag.

4.5 Extension ? deterministic rally-END via shuttle-floor contact (owner note, 2026-06-19) `[design]`

The same court geometry that localizes the prominent court also unlocks a **deterministic rally-END signal** ? the cheapest, most certain way to know a point is over: **the shuttle hit the floor.** This attacks **G3** (rally-END over-extension), where today's shuttle-velocity windowing runs the window long because "shuttle still moving ? rally over."

- **Introduce two geometric primitives:**
  - **The white court lines.** A badminton court is bounded by **white, straight lines forming a rectangle** (with the perspective trapezoid in image space). Detect them ? Hough lines / line-segment detection filtered to white, long, court-consistent segments ? to recover the **court quad** (which also *is* the prominent-court floor polygon of §4.2 route 4; one detector serves both).
  - **The floor plane.** The court lines define the **floor** (the plane the players stand on). The shuttle in flight lives *above* this plane; a shuttle *at* the plane (within the trapezoid, or just outside it) is **on the floor**.
- **The rally-END rule (deterministic):** a rally ends when the shuttle makes **floor contact** ? its trajectory descends to and **stops/rests at the floor level** (a near-zero-velocity terminus at the court-line plane), OR a shuttle that goes to the floor **outside the prominent court** = an out-of-bounds landing (also point-over). Unlike velocity heuristics, a shuttle resting on the floor is **unambiguous** ? the point is over.
- **Why it's strong + cheap:** no new model, no person stack ? it reuses the WASB trajectory (which already gives the shuttle's descending `(x, y)` + the `Visibility` drop when it settles) plus the court-line geometry. It is the **shuttle-side** rally-END signal, complementary to the **player-state** rally-END (player kinematics / held-shuttle) in `RALLY_DETECTION_GAP_CLOSURE.md` §4 Lever A ? fuse both for robustness (a shuttle can settle off-camera; a player can pick up early).
- **Subtlety to measure:** distinguish a true floor landing (point over) from a shuttle that **dips low but is played back up** (a tight net shot / low retrieve). The court-line plane + a brief **dwell** at floor level (the shuttle rests, doesn't re-accelerate up) is the discriminator; tune the dwell window on golden clips.

This is tracked as **C8** below and cross-links gap-closure **G3**. It shares the court-line detector with the prominent-court polygon (§4.2 route 4), so the two land together once court-line detection exists.

5. Experiment plan (measurement)

Land the gate flag-gated, then measure on the **cached trajectories** (CPU, no GPU/re-detect):
1. **GX010142 sanity:** draw a prominent-court polygon; confirm **rally #1 disappears** and the central rallies survive. The litmus the whole idea is born from.
2. **Multicourt golden** (`mahadevpura_2, GX010128, Badminton_BXH_2, Boxhill_Doubles`): ? rally-count, and once those clips are golden-labelled, ? precision / ? recall vs golden via `rally_seg_eval --stratify` (single vs multicourt). The win condition: multicourt precision up, **recall flat** (we drop FPs, not real rallies).
3. **Single-court no-op:** on single-court clips with no polygon (or a full-frame polygon) the gate is a verified no-op ? assert zero change.
4. **Dual-eval discipline** ([[eval-dual-set-regression]]): no per-video regression on either eval set.

Promotion to default-ON is a **separate, owner-gated** decision with a Launch Report (both rulers + the over-seg guard), never bundled with the experiment PR.

6. Risks & limits

- **Polygon is camera-dependent.** A hand-drawn polygon doesn't generalize across setups ? route 2/3 (auto-derivation) is the real product path; v1 config-polygon is for *validation*.
- **Shuttle height** (§4.3): too-tight a floor clips high clears; the pseudo-3D pad mitigates, measurement tunes it.
- **The prominent court can change** (camera re-aim mid-clip): out of scope v1 (assume static per clip).
- **Not a recall fix.** This only removes out-of-court FPs; it does nothing for missed in-court rallies (G1) ? and must be proven not to *cost* recall.

7. Open questions (owner / data)

1. Auto-derivation anchor: **player-location** (route 2) vs **shuttle-density** (route 3) as the default once validated? *(decides: data, post-experiment)*
2. Height-envelope: fixed upward pad vs per-camera perspective model ? is the fixed pad enough? *(decides: measurement)*
3. Do we expose the prominent court as a **one-time per-camera setup** (draw-once UI) or always auto? *(decides: owner/product)*

8. Deliverables & progress tracker ? source of truth

Legend: ? Todo · ? In progress · ? Done · ? Gated. One small PR per row, off `origin/master`, no stacking.

ID	Deliverable	Depends on	Gated?	Status	PR
C0	This design doc	?	No	?	this PR
C1	`prominent_court_gate(points, polygon, height_pad)` + unit tests (pure, point-in-polygon)	?	No (default-OFF)	?	?
C2	Thread the gate into the windowing input behind a default-OFF flag (`court_polygon` + `gate_shuttle_to_court`)	C1	No (default-OFF)	?	?
C3	GX010142 litmus: rally #1 dropped, central rallies kept (measured)	C2	No	?	?
C4	Extract person features (`detections_per_frame`) on GX010142 ? player-cluster = prominent court; measure `shuttle_player_colocation` too	?	No	?	?
C5	Multicourt-golden measurement (? count now; ? precision/recall vs golden once labelled)	C2, labels	? owner labels	?	?
C6	Auto-derive the polygon (route 2 player-anchored)	C4	? owner	?	?
C7	Launch decision (default-ON) with a Launch Report	C3,C5	? owner	?	?
C8	**Court-line / floor detection** ? (a) auto court polygon, (b) **deterministic shuttle-floor-contact rally-END** (§4.5, targets G3)	?	? owner	?	?

9. Definition of done

- [ ] The gate exists, is **default-OFF**, unit-tested, and a verified no-op on single-court / no-polygon.
- [ ] GX010142 rally #1 (and the far-right adjacent-court band) is dropped while the central rallies survive `[measured]`.
- [ ] On the multicourt golden clips, multicourt **precision improves with recall flat** vs golden labels (no per-video regression).
- [ ] A winning configuration is flipped default-ON only via a Launch Report ? flip `Status ? DONE`, archive.

```
117
118     ## 10. References
119     **Code `[verified]`:** `fusion_features.py` (`point_in_polygon`, `person_in_court`),
`person_detector.py` (`detections_per_frame`), `config/models.py`
(`FusionConfig.court_polygon`), `eval/windowing.py`, `eval/rally_seg_eval.py` (`--stratify`).
120     **Docs:** [`RALLY_DETECTION_GAP_CLOSURE.md`](RALLY_DETECTION_GAP_CLOSURE.md) (G2),
[`MULTI_SIGNAL_FUSION_PLAN.md`](MULTI_SIGNAL_FUSION_PLAN.md) (Gate B / court mask),
[`EXPERIMENT_HARNESS.md`](EXPERIMENT_HARNESS.md),
[`QUALITY_ITERATIONS.md`](QUALITY_ITERATIONS.md).
121
122     ---
123
124     ### Changelog
125     - 2026-06-19 ? Created from the owner's GX010142 rally-#1 (background-court FP)
observation + the measured multicourt over-fire (G2). Validated the 2D/pseudo-3D need from the
cached trajectory; mapped the reuse onto the existing `court_polygon` + `point_in_polygon`
machinery.
126     - 2026-06-19 ? Added §4.5 (owner note): a **deterministic rally-END signal via shuttle-
floor contact** ? detect the white court lines ? the floor plane ? a shuttle resting on the
floor (or landing out-of-court) ends the rally (targets G3). Shares the court-line detector with
the prominent-court polygon; tracked as C8.
127
```

7. user (2026-06-19T16:59:19.592Z)

```
1       # Next steps (live) ? owned-model + multisport
2
3       **Updated:** 2026-06-13 (**rally-detection quality track**: fusion + experiment-harness
design docs merged,
4       owner-gated ? see the dedicated section below, the desktop-agent pick-up). Prior:
2026-06-10 (audit?M0?Gemini-battery session; later same day the **UI-alpha track opened**:
5       ADR-009 ratified ? KhelSutra Guru / khelsutra.guru, private `khelsutra` repo, local-
first delivery ? see
6       `docs/UI_ALPHA_EXECUTION_PLAN.md`; **PR-1 async jobs (#16) MERGED 2026-06-11, PR #87**;
**PR-2
7       upload/download MERGED 2026-06-11, PR #99** ? live-E2E-verified (3-part chunked upload
MD5 byte-identity +
8       compile?browser-download); next: PR-3 identity. Same day: the **8?9 quality sweep
#90?#98 landed** ? ruff/mypy/
9       coverage-floor CI lanes, SQLite conn-close fix, config unknown-key warning, hybrid-twin
collapse into
10      `trajectory_hybrid.py`, WSL tilde+abspath live-path fixes; suite 388 green; LOVO 0.626
re-verified exact).
11      **Authoritative roadmap:**
12      `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+ ADR-008 for topology). **Latest blow-by-
blow:** memory
13      `current-state.md` / `session-handoff.md`. This file = the prioritized "what to do
next," refreshed each session.
14
15      ## ? Next-session / GPU-box pickup (2026-06-15)
16      **M0 in-container quick wins are COMPLETE:** ship-gate provisional blocker (**#170**),
train/eval split guard (**#171**), the **doc-consistency sweep** (**#174** ? corrected 16
drifted docs vs the code), and the **P3 person-feature fusion litmus machinery is already
present + suite-green** (`backend/eval/fusion_compare.py` + `ablation.py`: person-driven ?F1 /
bootstrap CI / paired sign test; the real-golden `skipif` litmus waits only on GPU-extracted
data). ? CI is red repo-wide due to a **GitHub Actions minutes/billing outage** (account-level,
not code) ? work is local-gated + `--admin`-merged until Actions is restored (Settings ? Billing
? Actions minutes / spending limit).
17
18      **Immediate next step ? the real P3 LOVO measurement** (this box has a GPU +
`torch+cu124` + `cv2`; videos and trajectory CSVs are present, but the GPU-extracted golden
features + `.rallies.csv` labels are not yet here):
19      1. **[owner / GPU box]** `python -m backend.eval.fusion_golden --manifest
output/human_lovo_manifest.json --out-dir "%USERPROFILE%\OneDrive\rally-annotator\golden-
data\features\2026-06-15-n6"` ? produces `*_golden_features.json` for the 6 golden videos. (GPU
```

+ WSL + videos + trajectory CSVs + `.rallies.csv` labels are owner-side only.)
2. Artifacts land in the **shared cloud folder** `C:\Users\avidu\OneDrive\rally-annotator\golden-data\` (OneDrive auto-syncs both ways). Convention + workflow: **[`GOLDEN_DATA_SHARING.md`](GOLDEN_DATA_SHARING.md)**; tracked in **#175**.
3. **[CPU dev/agent session]** `python -m backend.eval.fusion_compare --features-dir <shared>\features\2026-06-15-n6 --record` + `python -m backend.eval.ablation --features-dir <shared>\... --granularity group` ? real person-feature ?F1; copy the JSONs into `output/` to also pass the `skipif` litmus tests (`tests/test_ablation.py::test_litmus_reproduces_gate_a`).

Also queued (owner-gated / pending input):
- Ship-gate target calibration ? **#172** (owner decision; needs corpus growth).
- Tests reorganization ? blocked on the owner's grouping metric (provide it ? an agent will do it).
- Restore CI ? the Actions outage is account-level (billing/minutes), not a repo/config bug.

Leaving (bulky / M2 ? no start without owner go): ActionFormer (M0.4), event-index DB migration (M0.1), Gemini-silver purge (M0.3), cost-to-Gen-2 benchmark + ByteTrack deprecation, multisport, repo rename, PMF execution.

Where we are (one paragraph)
ADR-008 ratified ("repo split driven by productionization, not isolation"): training lives in-repo (`training/`)
behind a CI-enforced import wall; the featurizer is single-sourced (`backend/features/rally_seq.py`, train==serve);
the **Gen-0 harness ships** (`python -m training.gen0.harness --manifest output/human_lovo_manifest.json --floor
eval_baselines/heuristic_lovo_n6.json --version <semver>`) and produced artifact v0.1.0 (LOVO **0.626**, floor passed,
sign test 5/6 wins p=0.109 ? honestly not significant, ship=False). The **n=6 owned corpus is in the collector**
(262 rallies, Proprietary, commercial export = owned data only after the ShuttleSet reclassification). The **Gemini
battery is done** (see table) ? the moat is quantified.

The quality table that now drives strategy (IoU 0.5 vs human golden labels)
Path	Mahadevpura (clean-ish)	Kushagra (multi-court)
Heuristic (velocity windowing)	0.657	0.157
Gen-0 owned head (LOVO, n=6)	0.744	0.561
Two-stage WASB+Gemini refine	0.83	?
Gemini wrapper (`gemini-flash-latest`)	**0.93**	**0.66**

Reading: clean footage is commoditized (serve it with Gemini for cents). **Multi-court drops the commodity
wrapper to 0.66 ? that is the owned model's ship target** (its FPs are background-game pickups + dead-time merges,
the exact contrast-metric confound). Gen-0 is 0.10 behind with only n=6 ? a corpus-growth-sized gap, not an
architecture-sized one. Two-stage refine = the cost-efficient serving path for LONG tapes (uploads candidate clips
only); wrapper 503-flakiness (observed all session) makes the provider-fallback registry load-bearing.
Baselines tracked in `eval_baselines/` (heuristic_lovo_n6.json, gemini_wrapper_2026-06-10.json).

Rally-detection quality track (design merged 2026-06-13, owner-gated; implementation in progress)
This is the track a desktop/local agent is slated to pick up. Design docs landed (PR #112 + session). All implementation is **owner-gated** (per explicit in the plan); each step = ONE PR off `master`.

Hardening loop (T1-T5 positive tests + audit + finale) COMPLETE (2026-06-14, PRs #156/#157/#160/#161/#163 + #165 + final records). See archived `docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md` (Living Log with every

pre/post/review) and `docs/archives/HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md` (full rules + STATE). Top-level now pointer stubs per #165 review. All 5 items + verification + archive done; clean slate.

- **Docs:** `docs/MULTI_SIGNAL_FUSION_PLAN.md` (shuttle + person + audio, one fusion layer) ·
  `docs/EXPERIMENT_HARNESS.md` (A/B + shadow + promotion gate, zero-regression) ·
  `docs/ROORA_LABELING_GUIDELINES.md` (v8) · `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model).
- **Key finding (no re-investigation needed):** the owner's "held-shuttle counted as a rally" bug is **NOT a missing capability** ? `backend/pipeline/detectors/rally_gate.py` already implements the net-crossing **"Gate A"** (`apply_gate(..., min_crossings=2)`) that kills service-feed/held-shuttle windows, but it is **only called from eval drivers** (and there was no `min_crossings` knob in `IndexingConfig`). **Wiring Gate A into the live segmenter (now via fusion_hybrid) was the single cheapest, highest-confidence quality win.** (P2 FusionSegmenter specifics: registry `fusion_hybrid`, **default-OFF**, seeds from substrate, persists `net_crossings` + optional person features, optional served Gate A/B via config knobs; adversarially reviewed; suite green. See `tests/test_served_gate_a_regression.py` for the honest finding that on production windowing Gate A is neutral/negative in some cases, hence kept opt-in with knob for safety, and the leaderboard revert note.)
- **Current status (2026-06-14):** P1 (person cue extraction to reusable `PersonDetector`) + P2 (FusionSegmenter + Gate A signal + served handoff gating) + experiment harness P1 DONE (adversarial review + NO-REGRESSION; suite green). **P3 next (this PR + follow-ups):** learned fusion (person features ? harness `compare` LOVO lift on the 6 golden videos; eager-person-compute optimisation); then P4 audio via hit_detector.
- **Pick-up order (each = ONE PR off `master`, owner approval first, flag-gated default-OFF, promote only on measured LOVO):**
  1. (P2 first-step, largely done via fusion_hybrid) Wire Gate A into production served path + config knob (min_crossings); apply in hybrids at runtime. (Now available opt-in; served neutral on production windowing per litmus ? kept opt-in with knob for safety.)
  2. P3 first step (this work item): harness `compare` LOVO litmus for person-on variant (players_in_court + two_sided + colocation etc. from `fusion_features`) on the 6 golden; add the eager compute optimisation note/landing. Pure + harness (testable here).
  3. P3/P4 continuation: audio cue integration + full learned promotion if lift proven.
- **Discipline (from EXPERIMENT_HARNESS.md):** every new cue **flag-gated, default OFF**; promote to default **only on measured LOVO lift** through `run_regression.py` (exit 0/1/3); pinned `CONTROL` ? rollback = config flip. Harness re-uses ~80% existing code.
- ? Gate A, person cue logic, harness, and pure fusion paths are testable here; the heavier real-video / golden LOVO / ablation confirmation waits on the GPU-extracted golden features + `.rallies.csv` labels, which are not yet present in this checkout.
- **External due-diligence review inputs (2026-06-17) ? PICK UP AFTER current ideas are exhausted; full details + citations in [`RALLY_QUALITY_RESEARCH.md`](RALLY_QUALITY_RESEARCH.md) §5.6.** An owner-commissioned 31-reference review of the quality report *validated* the methodology and surfaced these **future** levers (research inputs, not active work, not new claims; verify citations before any submission):
  1. **"First-mile" paper framing (highest-value):** stroke-classifiers (BST/DSTA) + the big datasets (ShuttleSet ~36k strokes, Fine-Badminton ~27k actions) all **assume a pre-trimmed rally clip already exists** ? we solve the ignored prerequisite. Pitch the paper as *"the missing untrimmed-video preprocessing pipeline that lets BST/MonoTrack run on raw footage."*
  2. **Ground R2 in Action-Spotting / Precise-Event-Spotting** (SoccerNet/TenniSet/T-DEED) ? turns R2 into a defensible publishable contribution.
  3. **Add domain baselines for any submission:** MonoTrack + BST on ShuttleSet / Fine-Badminton (we have only heuristic + Gemini today).
  4. **Prefer TemporalMaxer over ActionFormer** for the M0.4 scorer swap (parameter-free max-pool, ~2.8× fewer GMACs, small-N robust) ? see "M0.4 next increment" below.
  5. **Court polygons ? pose ? the rally-END lever:** masking on-court players can resolve *occluded end-of-rally states* (the #1 remaining gap vs Gemini), not just spectator noise.
  6. **Name the eval?serve lesson "pipeline distribution skew"** ? a citable paper "lesson."

Discipline reminder: owner approval before starting any owner-gated item; one PR per step; babysit the PR (poll/fix/reply) until merge observed, *then* advance.

Prioritized next steps

1. **[owner, in progress] GROW THE GOLDEN CORPUS** ? the single highest-leverage move; every lever below feeds on it.
   Priority order for new videos: **(a) multi-court badminton** (the target distribution AND where the ship target
   lives), **(b) ?3 matches per new sport ? table tennis first** (WASB transfers at F1 0.909; pickleball labels are
   corpus-gold but not trainable until a clean MIT/Apache ball detector exists), (c) capture variety per
   `VIDEO_RECORDING_GUIDELINES.md`; **adult sessions only for the training pool** (consent guardrail). Per video:
   label ? `curate import-local --normalize --license Proprietary --fps <true fps>` ? re-run the Gen-0 harness.
   The paired sign test needs n: at n=10 with 9 wins, p?0.011 ? significance is reachable this month.
2. **[owner decision] Set the ship-gate target** ? now informed: floor = heuristic 0.480 (necessary), **multi-court
   reference = wrapper 0.66** (the number that makes the owned model worth serving), clean-footage ceiling = 0.93
   (not the target ? Gemini serves that tier). Suggested shape: "LOVO F1@tIoU0.5 ? 0.70 on multi-court folds with
   paired-sign p<0.05 vs heuristic" ? owner to ratify/adjust.
3. **M0.4 next increment ($0/local):** swap the Gen-0 LogReg for a lightweight temporal head (MIT, minutes on the
   2070S ? NOT the heavyweight Gen-2 stack) inside the existing harness; per-video threshold calibration (the
   LOVO-vs-oracle gap is visible per fold in the harness output). **Prefer TemporalMaxer** (parameter-free
   max-pool, ~2.8× fewer GMACs, small-N robust) over ActionFormer/ASFormer per the 2026-06-17 external review
   (`RALLY_QUALITY_RESEARCH.md` §5.6) ? better fit for the cheap/local/fast mandate.
4. **Multisport thread:** ingest Roora pickleball/TT CSVs (`import-local --normalize`); merged-prototype LOVO across
   badminton+TT once TT has ?3 matches; identify a clean pickleball ball detector. ? TT-on-WASB serving stays gated
   by C3 (provenance multiplies per sport).
5. **PMF validation (cheaper than any engineering month):** C13 academy interviews (+ the two added questions:
   "multi-court footage nobody watches?" / "pay per-match to index it?"); camera pilot at the warmest 2-3 academies
   (consent at capture; demo served via the Gemini path or human-polish ? the reel pattern exists:
   `output/MahadevpuraSingles_highlights.mp4`). **Field kit COMPLETE (2026-06-12, owner ratified the
   physical-visit approach; reframed product-feedback-first per owner spec):**
   `docs/PRODUCT_FIELD_ASSOCIATE_FLYER.md` (the recruiting flyer ? owner fills ?/contact placeholders and recruits) +
   `docs/FIELD_VISIT_PLAYBOOK.md` (the hired associate's visit playbook: coach + student script, demo-after-Q6
   rule, ground rules, Phase-2 record-and-show-back stretch goal) + `docs/FIELD_VISIT_ANSWER_SHEET.md` (per-visit
   capture form incl. demo-reaction + student blocks) + `docs/PMF_FIELD_SIGNALS.md` (G/A/R anchors + **pre-registered
   decision rules** ? PROCEED / PIVOT A-C / FALSIFIED thresholds fixed before the first visit).
6. **Housekeeping:** ~~rotate the Gemini API key~~ **DONE 2026-06-10** (owner-verified working; confirm old-key
   deletion in AI Studio); **rename repo off "badminton"** (owner action,
   in BACKLOG); collector=SoR confirmation; off-peak niceties: none pending ? the battery completed.

Pointers / artifacts

- Roadmap: `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` · Topology: ADR-008 (`docs/archives/decisions/DECISIONS.md`) ·
  Study: `docs/archives/research/OWNED_MODEL_TRAINING_STUDY.md` · Quality history:
  `docs/archives/quality-iterations/2026-06-multivideo-lovo/`.

```
118     - Harness: `training/gen0/harness.py` (one command ? train ? LOVO ? gate ? artifact
bundle; weights gitignored,
119       manifest reviewable). Featurizer: `backend/features/rally_seq.py` (FEATURE_SCHEMA v1).
120     - Corpus: collector DB (system-of-record, 6 matches/262 rallies) · eval manifest
`output/human_lovo_manifest.json`
121       (gitignored) · durable trajectories + Roora files: `C:\Users\avidu\Projects\Annotation
Setup\Collect\Trajectories\` ·
122       labels: `...\Collect\Golden Labelled\`.
123     - Gemini ops learnings: serial uploads (`indexing.ai_max_workers=1`) + chunk re-encode
124       (`indexing.ai_chunk_scale_width=1280`) + 6-attempt exponential generate retry ? a
cold/prepaid project burst-throttles
125       with a MISLEADING "credits depleted" 429; ramp gradually.
126     - ? No owned-model implementation beyond greenlit M0.x until owner greenlights the next
milestone. Committed next
127       PLATFORM slice = async job model (single-tenant).
128
```

8. assistant (2026-06-19T16:59:23.418Z)

Now let me look at the actual code: the windowing logic, the court gate / point_in_polygon, and the eval scorer to understand how recall/precision are actually measured.

9. user (2026-06-19T16:59:23.872Z)

```
1       """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3       **The bug this closes.** The offline eval substrate (``distill_local.build_candidates``
?
4       ``fusion_golden`` ? the golden feature JSONs the ablation/compare path re-scores) was
built
5       on a *recall-oriented PROPOSAL* windowing (``vt=0.005, merge_gap=1.0, min_window=0.5``):
6       deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
pipeline
7       (``TrajectoryHybridSegmenter.process_video``) windows COARSELY (``vt=0.02,
merge_gap=2.0,
8       min_window=2.0`` ? the ``config.indexing`` defaults). A cue measured on the proposal set
is
9       measured on candidates **production never serves**, so a "win" need not transfer. This
is
10      exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served
set
11      (``scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md`` vs ``output/gateA_served_verify/``).
12
13      **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
the
14      SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**
15      (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
16      build candidates under either preset, and :func:`served_windowing_from_config` lifts the
17      *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18      operating point a given deployment truly serves.
19
20      The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21      windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22      ``per_user_gate``). This module is the **what windowing**; that one is the **promotion
gate**.
23
24      Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25      caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26      """
27      from __future__ import annotations
```

```

    from dataclasses import dataclass
    from typing import Any, Dict, List, Mapping, Tuple

    from backend.config.models import IndexingConfig

    Interval = Tuple[float, float]

    __all__ = [
        "WindowingPreset",
        "SERVED",
        "PROPOSAL",
        "PRESETS",
        "served_windowing_from_config",
        "build_windows",
        "windows_to_intervals",
    ]


    @dataclass(frozen=True)
    class WindowingPreset:
        """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.

        ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
        "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
        ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
        knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
        """

        name: str
        velocity_thresh: float
        merge_gap: float
        min_window_duration: float
        #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
        #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
        max_window_duration: float = 0.0
        #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
        #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.
        hard_cap: bool = False
        #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
tail back to
        #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
        inactivity_end: float = 0.0
        #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
min fast-fraction.
        inactivity_rally_thresh: float = 0.08
        inactivity_min_density: float = 0.34
        #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
        #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
        #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
        #: factor keeps it surgical (a normal long rally is left alone).
        blob_fallback: bool = False
```

```
        blob_factor: float = 4.0

        def as_tuple(self) -> Tuple[float, float, float]:
            """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
            ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
            bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
            return (self.velocity_thresh, self.merge_gap, self.min_window_duration)

        def to_dict(self) -> Dict[str, Any]:
            return {
                "name": self.name,
                "velocity_thresh": self.velocity_thresh,
                "merge_gap": self.merge_gap,
                "min_window_duration": self.min_window_duration,
                "max_window_duration": self.max_window_duration,
                "hard_cap": self.hard_cap,
                "inactivity_end": self.inactivity_end,
                "inactivity_rally_thresh": self.inactivity_rally_thresh,
                "inactivity_min_density": self.inactivity_min_density,
                "blob_fallback": self.blob_fallback,
                "blob_factor": self.blob_factor,
            }


    def _served_preset_from_defaults() -> WindowingPreset:
        """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.

        ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
          - velocity:   ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
          - merge_gap:  ``idx_cfg.dead_time_merge_gap`` (default 2.0)
          - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)

        Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
        point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
        so the two can never silently diverge again.
        """
        idx = IndexingConfig()  # construct with defaults ? the served operating point
        return WindowingPreset(
            name="served",
            velocity_thresh=float(idx.wasb.velocity_threshold),  # == fusion/tracknet
default 0.02
            merge_gap=float(idx.dead_time_merge_gap),
            min_window_duration=float(idx.min_action_window_duration),
            max_window_duration=float(idx.max_window_duration),  # W1 bounded-windowing
(default 0=OFF)
            hard_cap=bool(idx.window_hard_cap),                  # W1 hard-cap fallback
(default OFF)
            inactivity_end=float(idx.window_inactivity_end),     # R4 rally-end trim
(default 0=OFF)
            inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
            inactivity_min_density=float(idx.inactivity_min_density),
            blob_fallback=bool(idx.window_blob_fallback),        # R5 blob-fallback
(default OFF)
            blob_factor=float(idx.window_blob_factor),
        )


    #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
```

```
cue is
    #: only promotable if it holds up HERE ? the operating point real users get.
    SERVED: WindowingPreset = _served_preset_from_defaults()

    #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
    #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
    #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
    #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
    PROPOSAL: WindowingPreset = WindowingPreset(
        name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
    )

    #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
    PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}


    def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
        """The SERVED windowing a *specific* live config serves (overrides honoured).

        Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
        arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
        (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
        default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
        this is *this config's* operating point. Falls back to the served defaults for any
absent key.
        """
        idx = _as_mapping(config, "indexing")
        family_cfg = _as_mapping(idx, family)
        return WindowingPreset(
            name=f"served:{family}",
            velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
            merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
            min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
            max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
            hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
            inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
            inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
SERVED.inactivity_rally_thresh)),
            inactivity_min_density=float(idx.get("inactivity_min_density",
SERVED.inactivity_min_density)),
            blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
            blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
        )


    def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
        """``obj[key]`` as a plain mapping, whether ``obj`` is a dict or a typed config
section
        (``AppConfig.get`` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
        if obj is None:
            return {}
        if isinstance(obj, Mapping):
            sub = obj.get(key, {})
        elif hasattr(obj, "get"):
```

```
180             sub = obj.get(key, {})
181         else:
182             sub = getattr(obj, key, {})
183         return sub if isinstance(sub, Mapping) else {}
184
185
186     def build_windows(points: List[Any], fps: float, frame_width: float,
187                       preset: WindowingPreset) -> List[Dict[str, Any]]:
188         """Trajectory ``points`` ? candidate windows under ``preset`` (the one windowing
call site).
189
190         A thin wrapper over ``TrackNetRunner.trajectory_to_action_windows`` that takes a
named preset
191         instead of three loose floats, so eval candidate-building and the served path
reference the
192         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
193         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195         vt, mg, mwd = preset.as_tuple()
196         return TrackNetRunner.trajectory_to_action_windows(
197             points, fps=fps, frame_width=frame_width,
198             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199             max_window_duration=preset.max_window_duration,
200             window_hard_cap=preset.hard_cap,
201             window_inactivity_end=preset.inactivity_end,
202             inactivity_rally_thresh=preset.inactivity_rally_thresh,
203             inactivity_min_density=preset.inactivity_min_density,
204             window_blob_fallback=preset.blob_fallback,
205             window_blob_factor=preset.blob_factor)
206
207
208     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209         """``[{start, end, ...}]`` ? ``[(start, end)]`` ? the interval shape the scorer/gate
speak."""
210         return [(w["start"], w["end"]) for w in windows]
211
```

10. user (2026-06-19T16:59:24.569Z)

```
1       """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3       WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4       the opponent for service after losing a point, that single trajectory looks like a
5       rally. The discriminator is structural, not duration-based: a real rally has the
6       shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7       single service feed crosses ~once.
8
9       This module is camera-agnostic and needs NO new model ? it works on the trajectory
10      we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11      axis with the larger shuttle spread) and the net position (median shuttle coord on
12      that axis ? the shuttle clusters at the two ends and crosses the net region often,
13      so the median sits near the net), then counts sign changes of (coord - net) across
14      the visible points inside a candidate window, with a deadband to ignore jitter.
15
16      Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17      Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18      windows from trajectory_to_action_windows, or to fold into it later.
19      """
20      from __future__ import annotations
21
22      from dataclasses import dataclass
23      from typing import List, Optional, Sequence, Tuple
24
25      Interval = Tuple[float, float]
```

```

    @dataclass
    class GatedWindow:
        start: float
        end: float
        crossings: int
        visible_points: int
        kept: bool


    def _spread(vals: Sequence[float]) -> float:
        """Robust spread = p95 - p5 (ignores a few outlier detections)."""
        if len(vals) < 2:
            return 0.0
        s = sorted(vals)
        lo = s[max(0, int(0.05 * (len(s) - 1)))]
        hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
        return hi - lo


    def _median(vals: Sequence[float]) -> float:
        if not vals:
            return 0.0
        s = sorted(vals)
        n = len(s)
        return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])


    def estimate_play_axis(points) -> Tuple[str, float, float]:
        """Return (axis 'x'|'y', net_value, span) from visible points.

        Play axis = the image axis along which the shuttle travels most (larger spread).
        For a baseline camera that's Y (players top/bottom); for a side camera, X.
        """
        xs = [p.x for p in points if p.visible]
        ys = [p.y for p in points if p.visible]
        sx, sy = _spread(xs), _spread(ys)
        if sx >= sy:
            return "x", _median(xs), sx
        return "y", _median(ys), sy


    def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
        """Count sign changes of (coord - net) across visible points, ignoring the
        deadband zone near the net (so jitter at the net doesn't inflate the count)."""
        crossings = 0
        last_side = None
        for p in window_points:
            if not p.visible:
                continue
            v = (p.x if axis == "x" else p.y) - net
            if abs(v) < deadband:
                continue
            side = v > 0
            if last_side is not None and side != last_side:
                crossings += 1
            last_side = side
        return crossings


    def _points_in(points, fps: float, start: float, end: float):
        return [p for p in points if start * fps <= p.frame <= end * fps]


```

```
91      def gate_windows(points, windows: List[Interval], fps: float,
92                       min_crossings: int = 2, deadband_frac: float = 0.06,
93                       estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94          """Annotate each (start,end) window with its shuttle net-crossing count and a
95          keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96          axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98          `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99          `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100         per window ? same result, avoids the per-window re-estimation."""
101         axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102         deadband = deadband_frac * span
103         out: List[GatedWindow] = []
104         for (s, e) in windows:
105             wp = _points_in(points, fps, s, e)
106             c = count_net_crossings(wp, axis, net, deadband)
107             vis = sum(1 for p in wp if p.visible)
108             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                                    kept=c >= min_crossings))
110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                    min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118
```

11. assistant (2026-06-19T16:59:27.802Z)

Now let me look at the eval scorer (how recall/precision/F1 are computed ? does it allow many-to-one matching? double-counting?) and the fusion_features point_in_polygon.

12. user (2026-06-19T16:59:28.904Z)

```
1       """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
2       over-segmentation-sensitive metric set, honestly.
3
4       This is the harness the rally-quality project (``docs/RALLY_QUALITY_RESEARCH.md`` §4.5 /
§7)
5       measures every tier against. It closes the loop the over-merge finding exposed: plain
temporal
6       IoU-F1 is blind to merged mega-windows, so we report it **alongside** segmental F1@k,
the
7       segment-count ratio, and the explicit ``merge_split`` breakdown
(``segmentation_metrics``).
8
9       For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds
candidate
10      windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
SERVED
11      operating point and any future windowing the same way) ? scores the windows-as-rallies
against
12      the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI,
and can
13      diff two windowings with a paired sign-test + CI (the per-tier ``delta``).
14
15      GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each
video's
16      trajectory path + fps + frame_width; the GT rally intervals come from
17      ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label
```

```
files.
18
19      Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
20      """
21      from __future__ import annotations
22
23      import argparse
24      import json
25      import os
26      from typing import Any, Dict, List, Optional, Tuple
27
28      from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
29      from backend.eval.metrics import Interval
30      from backend.eval.windowing import PRESETS, SERVED, WindowingPreset
31
32      DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
33      DEFAULT_FEATURES_DIR = "output"
34      DEFAULT_IOU = 0.5
35
36      #: Long-rally lens threshold (s). A rally is "long" (an eye-catching highlight) when its
duration
37      #: (end ? start) exceeds this. The local detector over-fires, and its false positives
are mostly
38      #: SHORT (motion blips, background-court fragments on multicourt footage); filtering to
long rallies
39      #: strips most of those FPs and reveals the real product quality on the long-rally happy
path. Used
40      #: as the default split point for the stratified report (``--long-tau`` overrides it).
DURATION, not
41      #: shot-count, is the filter: golden ``shots_count`` is unlabeled and the no-AI path
reports it as
42      #: "Unknown", whereas duration is derived from the same start/end labels we already
trust.
43      LONG_RALLY_TAU = 5.0
44
45      #: Multicourt golden clips ? the court-type lookup for the stratified report. The golden
manifest
46      #: (``output/human_lovo_manifest.json``) is gitignored, so this committed set is the
AUDITABLE source
47      #: of the single-vs-multicourt strata, kept in sync with ``docs/GOLDEN_VIDEOS.md`` (the
clips tagged
48      #: "multicourt"). Multicourt footage carries background games on adjacent courts ? the
dominant
49      #: source of short false positives ? so we report it separately. A manifest entry MAY
carry an
50      #: explicit ``court_type`` field, which overrides this set (see :func:`court_type_for`).
51      MULTICOURT_CLIPS = frozenset({"mahadevpura_2", "GX010128", "Badminton_BXH_2",
"Boxhill_Doubles"})
52
53
54      def court_type_for(video: Dict[str, Any]) -> str:
55          """``'multicourt'`` or ``'single'`` for a golden video. Honors an explicit manifest
56          ``court_type`` field when present, else falls back to membership in
``MULTICOURT_CLIPS``
57          (keyed by ``name``), else ``'single'``."""
58          ct = video.get("court_type")
59          if isinstance(ct, str) and ct:
60              return ct
61          return "multicourt" if video.get("name") in MULTICOURT_CLIPS else "single"
62
63
64      def filter_by_min_duration(ivs: List[Interval], min_duration: float) -> List[Interval]:
65          """The long-rally lens: keep only intervals strictly longer than ``min_duration``
seconds.
66
```

```
67          ``min_duration <= 0`` is an EXACT passthrough (the default-OFF semantics ? every
real rally has
68          positive duration, so nothing is dropped). It's a FILTER, not a new metric: applied
identically
69          to predictions AND ground truth before any matching, so the tIoU and R2 paths stay
consistent."""
70          if min_duration <= 0.0:
71              return list(ivs)
72          return [iv for iv in ivs if (iv[1] - iv[0]) > min_duration]
73
74
75      # ---------------------------------------------------------------------- #
76      # Pure scoring core (no I/O ? unit-testable)
77      # ---------------------------------------------------------------------- #
78      def score_intervals(preds: List[Interval], gts: List[Interval],
79                          iou: float = DEFAULT_IOU,
80                          tau_start: Optional[float] = None,
81                          tau_end: Optional[float] = None,
82                          min_duration: float = 0.0) -> Dict[str, Any]:
83          """The full per-video metric row for predicted vs ground-truth rally intervals.
84
85          Pairs temporal-IoU P/R/F1 + boundary error (``metrics.score``) with the
86          over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
so a
87          merged mega-window is *visible* even when IoU-F1 isn't moved.
88
89          Also carries the **R2** task-faithful block (``tolerance_metrics``, the headline
going forward;
90          tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
strict-1:1
91          tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-production costs
precision) + the
92          start/end boundary-error medians + the over-seg guard counts. See
docs/R2_EVAL_METRIC_DESIGN.md.
93
94          ``min_duration`` > 0 applies the **long-rally lens**
(:func:`filter_by_min_duration`): BOTH preds
95          and GTs are filtered to rallies longer than that many seconds BEFORE any matching,
so the whole
96          row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact
passthrough).
97          """
98          from backend.eval import tolerance_metrics as tm
99          preds = filter_by_min_duration(preds, min_duration)
100         gts = filter_by_min_duration(gts, min_duration)
101         ts = tm.TAU_START if tau_start is None else tau_start
102         te = tm.TAU_END if tau_end is None else tau_end
103         sc = metrics.score(preds, gts, iou_threshold=iou)
104         f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
105         ms = segmentation_metrics.merge_split_report(preds, gts)
106         tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
107         be = tm.boundary_error_report(preds, gts)  # unconditional best-overlap (the
R4-aiming diagnostic)
108         osr = tm.over_seg_report(preds, gts)
109         return {
110             "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
111             "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
112             "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
113             "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
114             "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
115             "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
116             "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
117             # --- R2 (tolerance-match) ? the task-faithful headline block ---
118             "tau_start": ts, "tau_end": te,
```

```
119             "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
120             "tol_matched": float(len(tol_matches)),
121             "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
be["dend"]["abs_median"],
122             "split_count": osr["split_count"], "merge_count": osr["merge_count"],
123         }
124
125
126     _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate",
"segment_count_ratio",
127                  "precision", "recall",
128                  "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median",
"dend_abs_median")
129
130
131     def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
132         """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.
133
134         Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
135         leave-one-video-out discipline (windows within a video are correlated)."""
136         out: Dict[str, Any] = {"n": len(rows)}
137         for k in _AGG_KEYS:
138             vals = [r[k] for r in rows]
139             out[k] = eval_stats.mean_with_ci(vals) if vals else None
140         return out
141
142
143     # ------------------------------------------------------------------------ #
144     # Golden corpus I/O
145     # ------------------------------------------------------------------------ #
146     def load_golden(manifest_path: str = DEFAULT_MANIFEST,
147                     features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:
148         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
149         intervals (``<name>_golden_features.json`` ``gts``). Videos with no GTs are kept but
flagged."""
150         with open(manifest_path) as fh:
151             manifest = json.load(fh)
152         out: List[Dict[str, Any]] = []
153         for v in manifest:
154             feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
155             gts: List[Interval] = []
156             if os.path.exists(feat):
157                 with open(feat) as fh:
158                     gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or
[])]
159             entry: Dict[str, Any] = {"name": v["name"], "trajectory": v["trajectory"],
160                                      "fps": float(v["fps"]), "frame_width":
float(v["frame_width"]),
161                                      "gts": gts}
162             # Optional prominent-court gate inputs (multicourt G2; default absent ? the gate
is a no-op):
163             # a normalized court_polygon + optional frame_height + court_height_pad from the
manifest.
164             for k in ("court_polygon", "frame_height", "court_height_pad"):
165                 if k in v:
166                     entry[k] = v[k]
167             out.append(entry)
168         return out
169
170
171     def windows_for(video: Dict[str, Any], preset: WindowingPreset,
172                     min_crossings: int = 0) -> List[Interval]:
```

```
        """Parse a video's trajectory, window it under ``preset`` ? predicted rally
intervals.

        ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
        ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
this many
        times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
        from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
        windowing (W1) leaves behind.

        NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
would
        drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
        strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
        """
        from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
        points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
        wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
        ivs = windowing.windows_to_intervals(wins)
        # Prominent-court WINDOW gate (multicourt G2, default-OFF): when the video carries a
normalized
        # court_polygon, drop candidate rallies whose shuttle is MAJORITY outside the
prominent court ? a
        # background/adjacent-court rally. Window-level (not point-level): a background
rally's central
        # tail keeps a few in-court points, enough to hold a window, so point-clearing alone
is too leaky
        # (measured on GX010142 rally #1). No polygon ? no-op. See
docs/PROMINENT_COURT_DETECTION.md.
        poly = video.get("court_polygon")
        if poly and ivs:
            from backend.pipeline.detectors.court_gate import gate_intervals_to_court
            fw = float(video["frame_width"])
            fh = float(video.get("frame_height") or fw * 9.0 / 16.0)
            ivs = gate_intervals_to_court(
                ivs, points, poly, float(video["fps"]), fw, fh,
                min_in_court_frac=float(video.get("court_min_in_court_frac", 0.5)),
                height_pad=float(video.get("court_height_pad", 0.0) or 0.0))
        if min_crossings > 0 and ivs:
            from backend.pipeline.detectors.rally_gate import gate_windows
            gated = [(g.start, g.end)
                     for g in gate_windows(points, ivs, video["fps"],
min_crossings=min_crossings)
                     if g.kept]
            # Gating emptied a non-empty video ? keep the ungated windows.
            ivs = gated if gated else ivs
        return ivs


    #: One windowed video before scoring: (name, court_type, predicted intervals, GT
intervals).
    PredictedVideo = Tuple[str, str, List[Interval], List[Interval]]


    def predict_all(golden: List[Dict[str, Any]], preset: WindowingPreset,
                    min_crossings: int = 0) -> List["PredictedVideo"]:
        """Window every scorable golden video ONCE (the expensive trajectory-parse +
windowing step),
        returning ``(name, court_type, preds, gts)`` per video. Lets callers re-score the
same
        predictions under several lenses (e.g. the all-vs-long stratified report) without
```

```
re-windowing."""
222         out: List[PredictedVideo] = []
223         for v in golden:
224             if not v["gts"] or not os.path.exists(v["trajectory"]):
225                 continue
226             out.append((v["name"], court_type_for(v), windows_for(v, preset, min_crossings),
v["gts"]))
227         return out
228
229
230     def _score_predicted(predicted: List["PredictedVideo"], iou: float,
231                          tau_start: Optional[float], tau_end: Optional[float],
232                          min_duration: float) -> List[Dict[str, Any]]:
233         """Score pre-windowed videos into per-video rows (carrying ``name`` +
``court_type``)."""
234         rows: List[Dict[str, Any]] = []
235         for name, court, preds, gts in predicted:
236             row = score_intervals(preds, gts, iou, tau_start, tau_end, min_duration)
237             row["name"] = name
238             row["court_type"] = court
239             rows.append(row)
240         return rows
241
242
243     def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
244                  iou: float = DEFAULT_IOU, min_crossings: int = 0,
245                  tau_start: Optional[float] = None, tau_end: Optional[float] = None,
246                  min_duration: float = 0.0) -> Dict[str, Any]:
247         """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
248         rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block, plus
249         its ``court_type``. ``min_duration`` > 0 applies the long-rally lens (preds + GTs
filtered to
250         rallies longer than that many seconds) consistently across both metric paths."""
251         rows = _score_predicted(predict_all(golden, preset, min_crossings),
252                                 iou, tau_start, tau_end, min_duration)
253         return {"preset": preset.to_dict(), "iou": iou, "min_crossings": min_crossings,
254                 "min_duration": min_duration, "rows": rows, "aggregate": aggregate(rows)}
255
256
257     def compare(golden: List[Dict[str, Any]], base: WindowingPreset, cand: WindowingPreset,
258                 iou: float = DEFAULT_IOU, base_min_crossings: int = 0,
259                 cand_min_crossings: int = 0,
260                 tau_start: Optional[float] = None, tau_end: Optional[float] = None,
261                 guard_weight_merge: Optional[float] = None,
262                 guard_weight_split: Optional[float] = None,
263                 guard_use_rates: bool = True, min_duration: float = 0.0) -> Dict[str, Any]:
264         """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?
(+ CI +
265         sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
266         over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
267         the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
268         alongside so the refinement is auditable. ``min_duration`` > 0 scores both sides
through the
269         long-rally lens (same filter applied to base and cand ? it's a scoring lens, not a
windowing
270         knob, so it is identical on both sides of the A/B)."""
271         from backend.eval import tolerance_metrics as tm
272         b = evaluate(golden, base, iou, base_min_crossings, tau_start, tau_end,
min_duration)
273         c = evaluate(golden, cand, iou, cand_min_crossings, tau_start, tau_end,
```

```
min_duration)
        by_name_b = {r["name"]: r for r in b["rows"]}
        paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
        delta: Dict[str, Any] = {}
        for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate",
"segment_count_ratio"):
            delta[k] = eval_stats.paired_delta_ci([rb[k] for rb, _ in paired],
                                                  [rc[k] for _, rc in paired])
        gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
        if guard_weight_merge is not None:
            gkw["weight_merge"] = guard_weight_merge
        if guard_weight_split is not None:
            gkw["weight_split"] = guard_weight_split
        guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)
        return {"base": b, "cand": c, "n_paired": len(paired), "delta": delta,
"over_seg_guard": guard}


    def stratified_report(golden: List[Dict[str, Any]], preset: WindowingPreset,
                          iou: float = DEFAULT_IOU, min_crossings: int = 0,
                          tau_start: Optional[float] = None, tau_end: Optional[float] =
None,
                          long_tau: float = LONG_RALLY_TAU) -> Dict[str, Any]:
        """The long-rally lens payoff: ``{all vs long(>long_tau)} × {single vs multicourt}``
aggregates.

        The thesis: local OVER-FIRES, and its false positives are mostly SHORT (motion
blips,
        background-court fragments on multicourt footage). Filtering to long rallies should
strip those
        FPs ? precision (``tol_precision``) should jump, most on multicourt. This table is
how we read
        "do we beat Gemini on the single-match long-rally happy path?".

        Windowing is run ONCE per video (``predict_all``); each cell re-scores the same
predictions
        under its duration lens. A cell aggregates only videos with >=1 golden rally in that
stratum
        (``num_gt`` > 0): a clip with no long rally is not evidence about long-rally quality
and its
        0/0 recall would distort the mean. ``n`` per cell reports the videos that survived
that gate."""
        predicted = predict_all(golden, preset, min_crossings)
        long_label = f"long>{long_tau:g}s"
        strata = (("all", 0.0), (long_label, long_tau))
        courts = ("single", "multicourt", "all")
        cells: Dict[Tuple[str, str], Dict[str, Any]] = {}
        for dur_label, min_dur in strata:
            rows = _score_predicted(predicted, iou, tau_start, tau_end, min_dur)
            for court in courts:
                sub = [r for r in rows
                       if (court == "all" or r["court_type"] == court) and r["num_gt"] > 0]
                cells[(dur_label, court)] = aggregate(sub)
        return {"long_tau": long_tau, "labels": [s[0] for s in strata], "courts":
list(courts),
                "cells": cells, "preset": preset.to_dict(), "iou": iou, "min_crossings":
min_crossings}


    # ------------------------------------------------------------------------ #
    # Reporting
    # ------------------------------------------------------------------------ #
    def _ci(m: Optional[Dict[str, Any]]) -> str:
        return "?" if not m else f"{m['mean']:.3f} [{m['ci_low']:.3f},{m['ci_high']:.3f}]"

```

```
325
326     def _format_guard(g: Dict[str, Any]) -> str:
327         """Render the R1 net over-seg promotion-guard verdict (with the strict rule for
contrast)."""
328         basis = "rate" if g["use_rates"] else "count"
329         lines = [f"\n## R1 over-seg promotion guard ({basis}-based, "
330                  f"w_merge={g['weight_merge']}, w_split={g['weight_split']})",
331                  f"  NET verdict: {'PASS' if g['passes'] else 'BLOCK'}  "
332                  f"(net cost {g['base_net']:.3f} ? {g['cand_net']:.3f}, ?
{g['net_delta']:+.3f}; "
333                  f"no_merge_rate_regress={g['no_merge_rate_regress']})",
334                  f"  strict per-video rule (R2 §2.3.3, for contrast): "
335                  f"{'pass' if g['strict_passes'] else 'BLOCK'}  "
336                  f"(count increases ? merges:{len(g['new_merges'])}
splits:{len(g['new_splits'])})"]
337         if g["merge_rate_regress"]:
338             names = ", ".join(f"{r['name']}({r['base']:.2f}?{r['cand']:.2f})"
339                               for r in g["merge_rate_regress"])
340             lines.append(f"  ? merge_rate REGRESSED on: {names}")
341         return "\n".join(lines)
342
343
344     def format_report(result: Dict[str, Any]) -> str:
345         p = result["preset"]
346         bounded = []
347         if p.get("max_window_duration"):
348             bounded.append(f"cap={p['max_window_duration']}" + ("(hard)" if
p.get("hard_cap") else ""))
349         if p.get("inactivity_end"):
350
bounded.append(f"inact_end={p['inactivity_end']}/rt={p['inactivity_rally_thresh']}"
351                            f"/md={p['inactivity_min_density']}")
352         if p.get("blob_fallback"):
353             bounded.append("blob_fallback")
354         extra = (", " + ", ".join(bounded)) if bounded else ""
355         md = result.get("min_duration", 0.0) or 0.0
356         md_tag = f", LONG-RALLY LENS min_dur>{md:g}s" if md > 0 else ""
357         lines = [f"# Rally-segmentation eval ? windowing '{p['name']}' "
358                  f"(vt={p['velocity_thresh']}, merge_gap={p['merge_gap']}, "
359                  f"min_win={p['min_window_duration']}{extra}), IoU={result['iou']}{md_tag}",
""]
360         lines.append("| video | F1 | F1@0.25 | F1@0.5 | merges | merge_rate | seg_ratio |
pred/gt |")
361         lines.append("|---|---|---|---|---|---|---|---|")
362         for r in result["rows"]:
363             lines.append(f"| {r['name']} | {r['f1']:.3f} | {r['f1@0.25']:.3f} |
{r['f1@0.5']:.3f} | "
364                          f"{int(r['merges'])} | {r['merge_rate']:.2f} |
{r['segment_count_ratio']:.2f} | "
365                          f"{r['num_pred']}/{r['num_gt']} |")
366         a = result["aggregate"]
367         lines += ["", f"**Aggregate (n={a['n']}, mean [95% CI]):** "
368                   f"F1 {_ci(a['f1'])} · F1@0.25 {_ci(a['f1@0.25'])} · F1@0.5
{_ci(a['f1@0.5'])} · "
369                   f"merge_rate {_ci(a['merge_rate'])} · seg_ratio
{_ci(a['segment_count_ratio'])}"]
370         # --- R2 (tolerance-match) block ? the task-faithful headline (tIoU above is
secondary) ---
371         rows = result["rows"]
372         if rows and "tol_f1" in rows[0]:
373             ts, te = rows[0]["tau_start"], rows[0]["tau_end"]
374             lines += ["", f"## R2 ? tolerance-match (?_start={ts}s, ?_end={te}s, strict
1:1)", "",
375                       "| video | tolF1 | tolP | tolR | |?start| | |?end| | split | merge |",
376                       "|---|---|---|---|---|---|---|---|"]
```

```
            for r in rows:
                lines.append(f"| {r['name']} | {r['tol_f1']:.3f} | {r['tol_precision']:.2f}
| "
                             f"{r['tol_recall']:.2f} | {r['dstart_abs_median']:.2f} | "
                             f"{r['dend_abs_median']:.2f} | {int(r['split_count'])} |
{int(r['merge_count'])} |")
            lines += ["", f"**R2 aggregate (n={a['n']}):** tolF1 {_ci(a['tol_f1'])} · "
                      f"tolP {_ci(a['tol_precision'])} · tolR {_ci(a['tol_recall'])} · "
                      f"|?start| {_ci(a['dstart_abs_median'])} · |?end|
{_ci(a['dend_abs_median'])} "
                      f"(start?solved / end=the gap)"]
        return "\n".join(lines)


    def format_stratified(strat: Dict[str, Any]) -> str:
        """Render the long-rally stratified table: {all vs long} × {single, multicourt, all-
courts}.

        ``tolP`` is the column that carries the story ? if filtering to long rallies strips
local's
        short false positives, precision rises (most on multicourt). ``n`` is the videos
with >=1 golden
        rally in that stratum."""
        long_tau = strat["long_tau"]
        cells = strat["cells"]
        lines = [f"\n## Long-rally lens ? stratified (long = duration > {long_tau:g}s; "
                 "n = videos with >=1 golden rally in the stratum)", "",
                 "| stratum | court | n | tolF1 | tolP | tolR | tIoU F1@0.5 | seg_ratio |",
                 "|---|---|---|---|---|---|---|---|"]
        for dur_label in strat["labels"]:
            for court in strat["courts"]:
                a = cells[(dur_label, court)]
                lines.append(f"| {dur_label} | {court} | {a['n']} | {_ci(a['tol_f1'])} | "
                             f"{_ci(a['tol_precision'])} | {_ci(a['tol_recall'])} | "
                             f"{_ci(a['f1@0.5'])} | {_ci(a['segment_count_ratio'])} |")
        return "\n".join(lines)


    def _resolve_preset(name: str) -> WindowingPreset:
        if name in PRESETS:
            return PRESETS[name]
        raise SystemExit(f"unknown preset '{name}'; choices: {sorted(PRESETS)}")


    def _build_preset(args: argparse.Namespace, side: str) -> WindowingPreset:
        """Resolve the ``base`` (``--preset``) or ``cand`` (``--vs``) windowing from
``args`` by
        applying the ``dataclasses.replace`` overrides for cap / hard-cap / blob-fallback /
        inactivity. ``cand`` falls back to the base override when its ``--vs-*`` variant is
unset
        (so a baseline-vs-candidate promotion check runs in one shot)."""
        from dataclasses import replace
        if side == "base":
            preset = _resolve_preset(args.preset)
            cap = args.max_window_duration
            hard_cap = args.hard_cap
            blob_fallback = args.blob_fallback
            inactivity_end = args.inactivity_end
        else:
            preset = _resolve_preset(args.vs)
            cap = (args.vs_max_window_duration if args.vs_max_window_duration is not None
                   else args.max_window_duration)
            hard_cap = args.vs_hard_cap or args.hard_cap
            blob_fallback = args.vs_blob_fallback or args.blob_fallback
            inactivity_end = (args.vs_inactivity_end if args.vs_inactivity_end is not None
```

```
                              else args.inactivity_end)
        if cap is not None:
            preset = replace(preset, max_window_duration=cap)
        if hard_cap:
            preset = replace(preset, hard_cap=True)
        if blob_fallback:
            preset = replace(preset, blob_fallback=True)
        if args.blob_factor is not None:
            preset = replace(preset, blob_factor=args.blob_factor)
        if inactivity_end is not None:
            preset = replace(preset, inactivity_end=inactivity_end)
        if args.inactivity_rally_thresh is not None:
            preset = replace(preset, inactivity_rally_thresh=args.inactivity_rally_thresh)
        if args.inactivity_min_density is not None:
            preset = replace(preset, inactivity_min_density=args.inactivity_min_density)
        return preset


    def main() -> None:
        ap = argparse.ArgumentParser(description="Rally-segmentation eval vs golden labels
(offline).")
        ap.add_argument("--manifest", default=DEFAULT_MANIFEST)
        ap.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR)
        ap.add_argument("--preset", default=SERVED.name, help="windowing preset (default:
served)")
        ap.add_argument("--vs", default=None, help="second preset to diff against --preset")
        ap.add_argument("--iou", type=float, default=DEFAULT_IOU)
        ap.add_argument("--min-crossings", type=int, default=0,
                        help="net-crossings rally-state gate on --preset (0=off; W2
Gate-A)")
        ap.add_argument("--vs-min-crossings", type=int, default=0,
                        help="net-crossings gate on --vs (default: same as --min-
crossings)")
        ap.add_argument("--max-window-duration", type=float, default=None,
                        help="override bounded-windowing cap (s) on BOTH presets (W1; e.g.
12)")
        ap.add_argument("--vs-max-window-duration", type=float, default=None,
                        help="cap (s) on --vs ONLY (default: same as --max-window-duration);
lets a "
                             "BASELINE (no cap) vs CANDIDATE (capped) promotion check run in
one shot")
        ap.add_argument("--hard-cap", action="store_true",
                        help="enforce the cap as a HARD cap on --preset (chop continuously-
active "
                             "windows at the cap when no inactivity gap exists; W1 hard-cap
fallback)")
        ap.add_argument("--vs-hard-cap", action="store_true",
                        help="hard-cap fallback on --vs (default: same as --hard-cap)")
        ap.add_argument("--blob-fallback", action="store_true",
                        help="R5 blob-fallback on --preset (after R4, force-chop any group
still over the "
                             "cap at its midpoint + flag/log it; the continuously-active
blob last resort)")
        ap.add_argument("--vs-blob-fallback", action="store_true",
                        help="R5 blob-fallback on --vs (default: same as --blob-fallback)")
        ap.add_argument("--blob-factor", type=float, default=None,
                        help="R5 magnitude gate (both presets): chop only groups > this ×
cap (default 4.0)")
        ap.add_argument("--inactivity-end", type=float, default=None,
                        help="R4 rally-end inactivity trim (s) on --preset (0=off; trims
post-rally drift tail)")
        ap.add_argument("--vs-inactivity-end", type=float, default=None,
                        help="R4 inactivity trim on --vs (default: same as --inactivity-
end)")
        ap.add_argument("--inactivity-rally-thresh", type=float, default=None,
```

```
                        help="R4 velocity above which motion counts as 'rally' (both
presets; default 0.08)")
        ap.add_argument("--inactivity-min-density", type=float, default=None,
                        help="R4 min trailing fast-fraction to stay 'in rally' (both
presets; default 0.34)")
        ap.add_argument("--tau-start", type=float, default=None,
                        help="R2 tolerance-match start window (s); default 2.0 (forgives the
service window)")
        ap.add_argument("--tau-end", type=float, default=None,
                        help="R2 tolerance-match end window (s); default 1.5 (keeps the
rally-end honest)")
        ap.add_argument("--guard-weight-merge", type=float, default=None,
                        help="R1 over-seg guard: merge cost weight (default 2.0; merge hides
a rally)")
        ap.add_argument("--guard-weight-split", type=float, default=None,
                        help="R1 over-seg guard: split cost weight (default 1.0; split only
fragments)")
        ap.add_argument("--guard-counts", action="store_true",
                        help="R1 over-seg guard scores raw counts instead of normalized
rates "
                             "(rates are the default ? comparable across a structural
mega?bounded change)")
        ap.add_argument("--min-duration", type=float, default=0.0,
                        help="LONG-RALLY LENS (0=OFF): score ONLY rallies longer than this
many seconds, "
                             "filtering BOTH predictions and golden GTs before matching.
Strips local's "
                             "short false-positive blips to reveal long-rally quality.
Applies across the "
                             "tIoU + R2 paths and to --vs (same filter on both sides).")
        ap.add_argument("--stratify", action="store_true",
                        help="also print the long-rally stratified report: {all vs
long(>--long-tau)} × "
                             "{single vs multicourt} aggregates (court type from
GOLDEN_VIDEOS.md / the "
                             "manifest court_type field). The actual long-rally payoff
table.")
        ap.add_argument("--long-tau", type=float, default=LONG_RALLY_TAU,
                        help=f"the 'long' rally duration threshold (s) used by --stratify "
                             f"(default {LONG_RALLY_TAU:g}; owner's North-star happy-path is
>5s).")
        ap.add_argument("--json-out", default=None, help="optional path to write the full
result JSON")
        args = ap.parse_args()

        golden = load_golden(args.manifest, args.features_dir)
        base = _build_preset(args, "base")
        if args.vs:
            vs_mc = args.vs_min_crossings or args.min_crossings
            cand = _build_preset(args, "cand")
            result = compare(golden, base, cand, args.iou,
                             base_min_crossings=args.min_crossings,
cand_min_crossings=vs_mc,
                             tau_start=args.tau_start, tau_end=args.tau_end,
                             guard_weight_merge=args.guard_weight_merge,
                             guard_weight_split=args.guard_weight_split,
                             guard_use_rates=not args.guard_counts,
                             min_duration=args.min_duration)
            print(format_report(result["base"]))
            print("\n" + format_report(result["cand"]))
            d = result["delta"]
            print(f"\n## ? ({args.vs} mc={vs_mc} ? {args.preset} mc={args.min_crossings}), "
                  f"n_paired={result['n_paired']}")
            for k, dd in d.items():
                st = dd["sign_test"]
```

```
533                 print(f"  ?{k}: {dd['mean_delta']:+.3f}
[{dd['ci_low']:+.3f},{dd['ci_high']:+.3f}] "
534                       f"CI?0={dd['ci_excludes_zero']} sign(W/L)={st['wins']}/{st['losses']}
p={st['p_value']:.3f}")
535             print(_format_guard(result["over_seg_guard"]))
536         else:
537             result = evaluate(golden, base, args.iou, args.min_crossings,
538                               tau_start=args.tau_start, tau_end=args.tau_end,
539                               min_duration=args.min_duration)
540             print(format_report(result))
541         if args.stratify:
542             # The long-rally payoff table always reports the BASE preset (the served/current
config),
543             # split by duration × court type. Independent of --min-duration (it does its own
split).
544             print(format_stratified(stratified_report(
545                 golden, base, args.iou, args.min_crossings,
546                 tau_start=args.tau_start, tau_end=args.tau_end, long_tau=args.long_tau)))
547         if args.json_out:
548             with open(args.json_out, "w", encoding="utf-8") as fh:
549                 json.dump(result, fh, indent=2)
550             print(f"\n[written {args.json_out}]")
551
552
553     if __name__ == "__main__":
554         main()
555
```

13. assistant (2026-06-19T16:59:32.095Z)

Now let me look at the existing court_gate (already shipped per #243) and the tolerance match scorer to understand the precision/recall mechanics precisely.

14. user (2026-06-19T16:59:33.057Z)

```
1       """Prominent-court shuttle gate ? multicourt G2 (see
``docs/PROMINENT_COURT_DETECTION.md``).
2
3       In multicourt footage the WASB shuttle tracker follows ANY shuttle in frame, so a rally
on a
4       background / adjacent court becomes a false rally on the filmed match. This gate re-
defines the
5       shuttle signal ``(x, y, visible)`` ? ``(x, y, visible AND inside-the-prominent-court)``:
a visible
6       shuttle point whose normalised position is outside the prominent-court polygon is marked
NOT
7       visible, so the velocity windower never opens a rally there.
8
9       Pure (no GPU / cv2): point-in-polygon over the court polygon (reuses
10      ``fusion_features.point_in_polygon``). **Subtractive only** ? it can only CLEAR
visibility, never
11      set it ? so on a correct polygon it suppresses out-of-court rallies without
manufacturing any.
12      ``polygon=None`` (or < 3 vertices, or non-positive frame dims) ? an EXACT no-op (single-
court /
13      unconfigured footage), matching ``rally_gate``'s do-no-harm discipline.
14
15      The polygon is the court FLOOR in normalised 0-1 image coords (same convention as
16      ``FusionConfig.court_polygon``). ``height_pad`` is the MVP pseudo-3D play-volume (§4.3):
a point up
17      to ``height_pad`` (normalised) ABOVE the floor ? i.e. higher in frame, smaller y ? is
still
18      admitted, so a high clear over the prominent court is not clipped.
19      """
20      from __future__ import annotations
```

```

    from dataclasses import replace
    from typing import Any, List, Optional, Sequence, Tuple

    from backend.pipeline.detectors.fusion_features import point_in_polygon

    Polygon = Sequence[Tuple[float, float]]


    def admitted(nx: float, ny: float, polygon: Polygon, height_pad: float = 0.0) -> bool:
        """Is the normalised point inside the court floor polygon, OR within ``height_pad``
above it?

        The pseudo-3D air column: shifting the point DOWN by ``height_pad`` (toward the
floor) and
        finding it inside means the original sits within ``height_pad`` above the floor ? a
high shuttle
        over the prominent court, admit it."""
        if point_in_polygon((nx, ny), polygon):
            return True
        return height_pad > 0.0 and point_in_polygon((nx, ny + height_pad), polygon)


    def in_court_fraction(interval: Tuple[float, float], points: List[Any], polygon:
Polygon,
                          fps: float, frame_width: float, frame_height: float,
                          height_pad: float = 0.0) -> float:
        """Fraction of a candidate rally's VISIBLE shuttle points that lie inside the
prominent court.

        ``interval`` is ``(start_s, end_s)``; points carry ``.frame``. Returns 1.0 when
there are no
        visible points in the interval (a window with no shuttle is not an out-of-court
false positive ?
        leave that judgement to the windower)."""
        if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
            return 1.0
        fs, fe = interval[0] * fps, interval[1] * fps
        n = inside = 0
        for p in points:
            if getattr(p, "visible", False) and fs <= p.frame <= fe:
                n += 1
                if admitted(p.x / frame_width, p.y / frame_height, polygon, height_pad):
                    inside += 1
        return 1.0 if n == 0 else inside / n


    def gate_intervals_to_court(intervals: List[Tuple[float, float]], points: List[Any],
                                polygon: Optional[Polygon], fps: float,
                                frame_width: float, frame_height: float,
                                min_in_court_frac: float = 0.5, height_pad: float = 0.0
                                ) -> List[Tuple[float, float]]:
        """Drop candidate rallies whose shuttle is MAJORITY out-of-court ? the window-level
gate.

        Point-level gating (:func:`gate_points_to_court`) is leaky: a background-court rally
whose shuttle
        occasionally drifts into the prominent court keeps a few in-court points, enough to
hold a (sparse)
        window (measured: GX010142 rally #1 is 90% adjacent-court yet a 10% central tail
survives a hard
        point gate). This drops an interval whose in-court point fraction is below
``min_in_court_frac``,
        so a rally that is *mostly* in the adjacent court is removed wholesale. No-op
```

```
(returns all
73          intervals) when ``polygon`` is None/degenerate. Subtractive-only."""
74          if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0 or fps
<= 0:
75              return intervals
76          return [iv for iv in intervals
77                  if in_court_fraction(iv, points, polygon, fps, frame_width, frame_height,
height_pad)
78                  >= min_in_court_frac]
79
80
81      def gate_points_to_court(points: List[Any], polygon: Optional[Polygon],
82                               frame_width: float, frame_height: float,
83                               height_pad: float = 0.0) -> List[Any]:
84          """Return ``points`` with visibility CLEARED for any visible shuttle outside the
prominent-court
85          polygon (expanded upward by ``height_pad``).
86
87          No-op (returns the input list unchanged) when ``polygon`` is None/degenerate or the
frame dims
88          are non-positive. Otherwise each gated point is a NEW ``TrajectoryPoint`` (the input
list and
89          its points are never mutated)."""
90          if not polygon or len(polygon) < 3 or frame_width <= 0 or frame_height <= 0:
91              return points
92          out: List[Any] = []
93          for p in points:
94              if getattr(p, "visible", False):
95                  nx, ny = p.x / frame_width, p.y / frame_height
96                  if not admitted(nx, ny, polygon, height_pad):
97                      p = replace(p, visible=False)
98              out.append(p)
99          return out
100
```

15. user (2026-06-19T16:59:33.363Z)

```
1       """R2 ? task-faithful rally-boundary eval (tolerance-match + over-segmentation guard).
2
3       The headline rally metric (tIoU-F1 is kept as a SECONDARY diagnostic): a prediction
matches a
4       golden rally iff ``|?start| ? ?_start AND |?end| ? ?_end`` under a **strict 1:1
assignment**, so
5       over-production costs precision directly. Reported as a DECOMPOSITION ? ``P/R/F1@?`` +
start/end
6       boundary-error distributions + an over-seg guard (split/merge counts) + a ?-sweep ?
never one
7       number (a single ? misleads; see the design doc).
8
9       Locked decisions + rationale: ``docs/R2_EVAL_METRIC_DESIGN.md`` (owner-reviewed
2026-06-17):
10      asymmetric ? (``?_start=2.0`` forgives the ~1?1.5 s service window; ``?_end=1.5`` keeps
the
11      rally-end honest ? IAA-calibrated later); augment tIoU now, ablation-gate any eventual
swap.
12
13      Pure + stdlib; **numpy/scipy are OPTIONAL** ? used for the exact Hungarian assignment
when present,
14      else a deterministic greedy 1:1 fallback. Both agree on the temporally-ordered, near-
diagonal
15      eligibility graphs that rally intervals produce, so scores are environment-independent.
16      """
17      from __future__ import annotations
18
19      import statistics
```

```
    from typing import Any, Dict, List, Optional, Tuple

    Interval = Tuple[float, float]
    #: (pred_idx, gt_idx, dstart_signed, dend_signed) ? signed = pred ? gt (positive = pred
is late).
    Match = Tuple[int, int, float, float]

    #: Provisional ? (the design's locked starting point; fixed by the IAA study later).
Always
    #: report the sweep alongside, since the golden END convention carries >1.5 s of noise.
    TAU_START: float = 2.0
    TAU_END: float = 1.5
    SWEEP_TAUS: Tuple[float, ...] = (1.5, 2.0, 2.5, 3.0)


    def _eligible(p: Interval, g: Interval, tau_start: float, tau_end: float) -> bool:
        return abs(p[0] - g[0]) <= tau_start and abs(p[1] - g[1]) <= tau_end


    def _greedy_match_1to1(preds: List[Interval], gts: List[Interval],
                           tau_start: float, tau_end: float) -> List[Match]:
        """Deterministic greedy 1:1: eligible (pred, gt) pairs in ascending combined
boundary error,
        assigned first-come and skipping used indices. Optimal-or-near on the near-diagonal
eligibility
        graph rally intervals produce. The numpy/scipy-free path (and the CI path)."""
        pairs: List[Tuple[float, float, int, int]] = []
        for i, p in enumerate(preds):
            for j, g in enumerate(gts):
                if _eligible(p, g, tau_start, tau_end):
                    # sort key: combined error, then (i, j) for a fully deterministic
tiebreak
                    pairs.append((abs(p[0] - g[0]) + abs(p[1] - g[1]), float(i * 1e-6 + j *
1e-9), i, j))
        pairs.sort()
        used_p: set = set()
        used_g: set = set()
        matches: List[Match] = []
        for _, _, i, j in pairs:
            if i in used_p or j in used_g:
                continue
            used_p.add(i)
            used_g.add(j)
            matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
        matches.sort(key=lambda mm: mm[1])
        return matches


    def _hungarian_match_1to1(preds: List[Interval], gts: List[Interval],
                              tau_start: float, tau_end: float) -> Optional[List[Match]]:
        """Exact max-cardinality (then min total boundary error) via scipy. Returns ``None``
when
        numpy/scipy are unavailable so the caller falls back to greedy."""
        try:
            import numpy as np
            from scipy.optimize import linear_sum_assignment
        except Exception:
            return None
        n, m = len(preds), len(gts)
        size = max(n, m)
        big = 1.0e6
        cost = np.full((size, size), big, dtype=float)
        for i, p in enumerate(preds):
            for j, g in enumerate(gts):
                if _eligible(p, g, tau_start, tau_end):
```

```
                    cost[i, j] = abs(p[0] - g[0]) + abs(p[1] - g[1])
        rows, cols = linear_sum_assignment(cost)
        matches: List[Match] = []
        for i, j in zip(rows.tolist(), cols.tolist()):
            if i < n and j < m and cost[i, j] < big:  # drop forced ineligible assignments
                matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
        matches.sort(key=lambda mm: mm[1])
        return matches


    def match_1to1(preds: List[Interval], gts: List[Interval],
                   tau_start: float = TAU_START, tau_end: float = TAU_END
                   ) -> Tuple[List[Match], float, float, float]:
        """Strict 1:1 tolerance match ? ``(matches, precision, recall, f1)``. ``P = matched
/ n_pred``
        (over-production costs precision), ``R = matched / n_gt``. Exact Hungarian when
scipy is
        present; deterministic greedy otherwise."""
        n, m = len(preds), len(gts)
        if n == 0 or m == 0:
            return [], 0.0, 0.0, 0.0
        matches = _hungarian_match_1to1(preds, gts, tau_start, tau_end)
        if matches is None:
            matches = _greedy_match_1to1(preds, gts, tau_start, tau_end)
        k = len(matches)
        precision = k / n
        recall = k / m
        f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) else
0.0
        return matches, precision, recall, f1


    def _percentile(xs: List[float], p: float) -> float:
        """Linear-interpolated percentile (pure stdlib)."""
        if not xs:
            return 0.0
        s = sorted(xs)
        k = (len(s) - 1) * p / 100.0
        lo = int(k)
        hi = min(lo + 1, len(s) - 1)
        return s[lo] + (s[hi] - s[lo]) * (k - lo)


    def _overlap(a: Interval, b: Interval) -> float:
        return max(0.0, min(a[1], b[1]) - max(a[0], b[0]))


    def boundary_error_report(preds: List[Interval], gts: List[Interval]) -> Dict[str,
Dict[str, float]]:
        """Median + IQR of signed AND absolute ?start, ?end, split start/end ? the "start
solved,
        end open" diagnostic that aims R4.

        Computed over **best-overlap** matches (each GT ? its maximally-overlapping
prediction, any
        positive overlap), NOT the within-? 1:1 matches: a ?-gated set is survivorship-
biased (loose
        ends fail the gate and vanish from the distribution), which would HIDE the very end-
deficit
        this diagnostic exists to surface. Unconditional best-overlap is what the day-1
validation used
        to find |?end| ? |?start|. (The within-? deficit shows up instead as low
``tol_recall``.)"""
        def dist(vals: List[float]) -> Dict[str, float]:
            av = [abs(v) for v in vals]
```

```
133             return {
134                 "signed_median": statistics.median(vals) if vals else 0.0,
135                 "abs_median": statistics.median(av) if av else 0.0,
136                 "abs_p25": _percentile(av, 25.0),
137                 "abs_p75": _percentile(av, 75.0),
138                 "n": float(len(vals)),
139             }
140         ds: List[float] = []
141         de: List[float] = []
142         for g in gts:
143             best = max(preds, key=lambda p: _overlap(p, g), default=None)
144             if best is not None and _overlap(best, g) > 0.0:
145                 ds.append(best[0] - g[0])
146                 de.append(best[1] - g[1])
147         return {"dstart": dist(ds), "dend": dist(de)}
148
149
150     def over_seg_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, int]:
151         """The over-segmentation GUARD (per-video no-regression in promotion):
``split_count`` (?2
152         preds covering one rally) + ``merge_count`` (one pred ? ?2 rallies). Reuses the
bipartite
153         overlap-graph in ``segmentation_metrics.merge_split_report``."""
154         from backend.eval.segmentation_metrics import merge_split_report
155         r = merge_split_report(preds, gts)
156         return {"split_count": int(r["splits"]), "merge_count": int(r["merges"])}
157
158
159     # ---------------------------------------------------------------------- #
160     # R1 ? net over-segmentation promotion guard
161     # ---------------------------------------------------------------------- #
162     #: Default over-seg cost weights. A MERGE hides a rally (?2 golden rallies collapse into
one
163     #: predicted window ? the user can't reach the hidden one ? recall loss at rally
granularity); a
164     #: SPLIT only fragments one rally (it is still found, just in pieces). So a merge is the
strictly
165     #: worse fault and is weighted higher. (Equal weights also pass the milestone ? see the
R1 evidence;
166     #: the asymmetry is the principled default, not a number tuned to force a verdict.)
167     WEIGHT_MERGE: float = 2.0
168     WEIGHT_SPLIT: float = 1.0
169
170
171     def over_seg_guard(base: List[Dict[str, Any]], cand: List[Dict[str, Any]], *,
172                        weight_merge: float = WEIGHT_MERGE, weight_split: float =
WEIGHT_SPLIT,
173                        use_rates: bool = True, eps: float = 1e-9) -> Dict[str, Any]:
174         """**Net** over-segmentation promotion guard (the R1 refinement of the strict per-
video rule).
175
176         ``base``/``cand`` are aligned per-video over-seg dicts (the rows ``rally_seg_eval``
already
177         emits): each needs ``split_count``/``merge_count`` and ? when ``use_rates``
(default) ?
178         ``merge_rate``/``split_rate``. Optional ``name`` keys align the two lists by video
(else by
179         position). Returns the promote-or-block verdict for promoting ``cand`` over
``base``.
180
181         **Why the strict rule needed refining.** R2 §2.3.3's guard blocks promotion if
``cand`` raises
182         ``split_count`` OR ``merge_count`` on ANY video. That is right for an *incremental*
cue (same
183         windowing structure, an increase = a real regression) but mis-fires on a
```

```
*structural* change
184         (mega-windows ? bounded): splitting a single mega-window necessarily lifts
``split_count`` from
185         ~0, and the raw merge **count** is non-monotonic ? one mega-window swallowing 40
rallies is
186         ``merge_count=1`` yet ``merge_rate=1.0`` (every rally hidden), whereas bounding it
into pieces
187         that each glue 2 neighbours is ``merge_count=9`` yet ``merge_rate=0.5`` (FEWER
rallies hidden).
188         So the strict count rule blocks a config that strictly *reduces* the fraction of
rallies lost to
189         over-merge. (Measured: the cap6+R4 milestone trips strict on 10/11 videos yet cuts
mean
190         ``merge_rate`` 0.694 ? 0.150 with NO per-video merge_rate regression ?
RALLY_QUALITY_RESEARCH §6.)
191
192         **The net rule (default verdict ``passes``):** score over-seg as ONE weighted cost
per video and
193         require BOTH:
194           1. the corpus-mean net cost does not rise (``cand_net ? base_net + eps``), AND
195           2. NO video's **merge_rate** rises beyond ``eps`` ? reintroducing a mega-window
that hides MORE
196              of a video's rallies is the one over-merge regression that is never acceptable,
the honest
197              "no NEW merges" rule expressed on the *rate* (the recall-meaningful quantity),
not the count.
198         With ``use_rates`` the per-video cost uses ``merge_rate``/``split_rate`` (fraction
of GT rallies
199         affected, ? [0,1] ? comparable across a structural change); ``use_rates=False``
scores raw counts.
200
201         **Revert (one line):** read ``strict_passes`` instead of ``passes`` to fall back to
the original
202         strict per-video count rule. Both verdicts are always reported, so the choice is
auditable.
203         """
204         def _aligned() -> List[Tuple[Dict[str, Any], Dict[str, Any]]]:
205             if base and cand and all("name" in b for b in base) and all("name" in c for c in
cand):
206                 cmap = {c["name"]: c for c in cand}
207                 return [(b, cmap[b["name"]]) for b in base if b["name"] in cmap]
208             n = min(len(base), len(cand))
209             return [(base[i], cand[i]) for i in range(n)]
210
211         def _cost(row: Dict[str, Any]) -> float:
212             if use_rates:
213                 return weight_merge * float(row["merge_rate"]) + weight_split *
float(row["split_rate"])
214             return weight_merge * float(row["merge_count"]) + weight_split *
float(row["split_count"])
215
216         pairs = _aligned()
217         n = len(pairs)
218         base_costs = [_cost(b) for b, _ in pairs]
219         cand_costs = [_cost(c) for _, c in pairs]
220         base_net = (sum(base_costs) / n) if n else 0.0
221         cand_net = (sum(cand_costs) / n) if n else 0.0
222         net_delta = cand_net - base_net
223
224         # Per-video regressions, reported for transparency.
225         new_merges: List[Dict[str, Any]] = []           # raw COUNT increases (what strict
trips on)
226         new_splits: List[Dict[str, Any]] = []           # raw COUNT increases (what strict
trips on)
227         merge_rate_regress: List[Dict[str, Any]] = []    # RATE increases (the real over-
```

```
merge regression)
        for b, c in pairs:
            if float(c["merge_count"]) > float(b["merge_count"]) + eps:
                new_merges.append({"name": b.get("name", ""),
                                   "base": float(b["merge_count"]), "cand":
float(c["merge_count"])})
            if float(c["split_count"]) > float(b["split_count"]) + eps:
                new_splits.append({"name": b.get("name", ""),
                                   "base": float(b["split_count"]), "cand":
float(c["split_count"])})
            if use_rates and float(c["merge_rate"]) > float(b["merge_rate"]) + eps:
                merge_rate_regress.append({"name": b.get("name", ""),
                                           "base": float(b["merge_rate"]), "cand":
float(c["merge_rate"])})

        strict_passes = (not new_merges) and (not new_splits)
        net_cost_ok = net_delta <= eps
        no_merge_rate_regress = not merge_rate_regress
        passes = bool(net_cost_ok and no_merge_rate_regress)
        return {
            "passes": passes,
            "strict_passes": strict_passes,
            "net_cost_ok": net_cost_ok,
            "no_merge_rate_regress": no_merge_rate_regress,
            "base_net": base_net,
            "cand_net": cand_net,
            "net_delta": net_delta,
            "n": n,
            "use_rates": use_rates,
            "weight_merge": weight_merge,
            "weight_split": weight_split,
            "new_merges": new_merges,
            "new_splits": new_splits,
            "merge_rate_regress": merge_rate_regress,
        }


    def tolerance_report(preds: List[Interval], gts: List[Interval],
                         tau_start: float = TAU_START, tau_end: float = TAU_END) ->
Dict[str, object]:
        """The full R2 block for one video: P/R/F1@? + boundary-error distributions + over-
seg guard."""
        matches, precision, recall, f1 = match_1to1(preds, gts, tau_start, tau_end)
        return {
            "tau_start": tau_start, "tau_end": tau_end,
            "n_pred": len(preds), "n_gt": len(gts), "matched": len(matches),
            "precision": precision, "recall": recall, "f1": f1,
            "boundary_error": boundary_error_report(preds, gts),
            "over_seg": over_seg_report(preds, gts),
        }


    def tolerance_sweep(preds: List[Interval], gts: List[Interval],
                        taus: Tuple[float, ...] = SWEEP_TAUS) -> Dict[float, float]:
        """Symmetric-? F1 sweep ? the trend curve (a single ? misleads). Monotonic non-
decreasing in ?."""
        return {t: match_1to1(preds, gts, t, t)[3] for t in taus}

```

16. assistant (2026-06-19T16:59:37.354Z)

I now have a precise understanding of the scoring mechanics. Let me check the cached det_raw.jsonl reference and the GX010142 measured data to confirm the multi-candidate claims, and look at whether there's a multi-target linker prototype anywhere.

17. user (2026-06-19T16:59:40.146Z)

```
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17/output/wasb/Boxh
ill_Doubles_proxy__9a35fffdfbde_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17/output/wasb/GX01
0132_proxy__430d77131262_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17/output/wasb/GX01
0135_proxy__f10da909f0a4_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17/output/wasb/GX01
0137_1080p__74eedbc35d73_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17/output/wasb/GX01
0138_1080p__1b24bb67717f_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17/output/wasb/GX01
0139_1080p__4229be621384_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17/output/wasb/GX01
0140_1080p__80cfa1d20990_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17/output/wasb/GX01
0141_1080p__daaf8af19864_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17/output/wasb/GX02
0094_proxy__08471270f285_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17/output/wasb/GX02
0140_1080p__d8230f6d57b1_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/hardchop/output/wasb/Badminton BXH
2_proxy__ebebccb702c4_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hardchop/output/wasb/GX030
094_proxy__27f1c707bbfb_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-
safeguard/output/wasb/GX010128__db84f3e65318_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-
safeguard/output/wasb/GX010129__99a3e73afde3_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/w2-
safeguard/output/wasb/mahadevpura-0__6460c2fc7647_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-indexer/output/wasb/Adarsh v
Avi__bb961d0d0cff_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-
indexer/output/wasb/GX010125_1080p__89605e80ed01_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-
indexer/output/wasb/GX010142_1080p__c4cfa94a792f_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-
indexer/output/wasb/GX010143_1080p__c991e53106c8_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-
indexer/output/wasb/GX010144_1080p__74affb36e15e_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-
indexer/output/wasb/GX010145_1080p__5a6fe8c350d4_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-
indexer/output/wasb/GX010147_1080p__79ae625eac6d_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-
indexer/output/wasb/mahadevpura_smoke_180s__05e0e577029d_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-
indexer/output/wasb/smoke_GX010143__1803d4c01a73_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-
indexer/output/wasb_sxs/png/mahadevpura_smoke_180s__05e0e577029d_wasb_wasbcache/det_raw.jsonl
/c/Users/avidu/Projects/badminton-highlight-
indexer/output/wasb_sxs/stream/mahadevpura_smoke_180s__05e0e577029d_wasb_wasbcache/det_raw.jsonl
---ARTIFACTS---
total 1012
drwxr-xr-x 1 avidu 197609      0 Jun 19 22:25 .
drwxr-xr-x 1 avidu 197609      0 Jun 19 21:46 ..
-rw-r--r-- 1 avidu 197609  18149 Jun 18 13:25 long_rally_lens_n15.json
-rw-r--r-- 1 avidu 197609   3870 Jun 18 13:25 long_rally_lens_n15.txt
drwxr-xr-x 1 avidu 197609      0 Jun 19 22:22 multishuttle_142
-rw-r--r-- 1 avidu 197609   5602 Jun 19 22:23 pr245_code_review.md
-rw-r--r-- 1 avidu 197609   4028 Jun 19 22:23 pr245_fixes_plan.md
-rw-r--r-- 1 avidu 197609   8520 Jun 19 22:25 pr245_review.json
```

```
-rw-r--r-- 1 avidu 197609   5909 Jun 19 22:22 pr245_review.md
drwxr-xr-x 1 avidu 197609      0 Jun 10 15:54 rally_tal_gen0
-rw-r--r-- 1 avidu 197609 912462 Jun 16 08:50 tridet.pdf
-rw-r--r-- 1 avidu 197609  46340 Jun 16 08:50 tridet.txt
---scratch which problem---
ABLATION_FRAMEWORK_PLAN.md
GATE_AB_NUMBERS_AND_IMPLICATIONS.md
HIGH_RISK_REMEDIATION_PLAN.md
OWNED_MODEL_PHASE0_PLAN.md
PERSONALIZATION_FEATURE_DESIGN.md
PERSONALIZATION_PIVOT_ANALYSIS.md
PR1_BODY.md
PR1_HANDOFF.md
PR2_HANDOFF.md
PR2_deferred_hardening_status.patch
PR88_FIX.md
Rally_Detection_Quality_Report.pdf
__pycache__
_final_p1.png
_final_p4.png
_fusion_ablation_chart.png
_pov_final_p2.png
_pov_p1.png
_pov_p2.png
_rev_p1.png
_rev_p2.png
_rev_p3.png
_rev_p4.png
_rev_p5.png
_rev_p6.png
_rev_p7.png
_zoom_chart.png
_zoom_tbl1.png
ab_eval_demo.py
ab_n6_lovo.py
ablation_report.pdf
analyze_19june.py
analyze_motion.py
analyze_multishuttle_142.py
arial.ttf
at_par.py
boxhill_highlights_stage
ci_commit_msg.txt
clean_state.py
compare_19june_report.py
compare_local_vs_gemini.py
debug_wsl.py
diagnose_rally.py
extract_19june.py
extract_multishuttle_frames_142.py
extract_newclips_until_done.ps1
extract_person_19june.py
extract_until_done.ps1
extract_val_until_done.ps1
extract_wrapper.log
```

18. user (2026-06-19T16:59:41.269Z)

```
No files found
```

19. assistant (2026-06-19T16:59:44.464Z)

There's a `multishuttle_142` artifact directory and analysis scripts that are the actual measurement basis for the design. Let me read the measured data and the analysis script ? this is the empirical foundation of the recall/precision claim.

20. user (2026-06-19T16:59:46.400Z)

```
total 14832
drwxr-xr-x 1 avidu 197609       0 Jun 19 22:22 .
drwxr-xr-x 1 avidu 197609       0 Jun 19 22:25 ..
-rw-r--r-- 1 avidu 197609    2072 Jun 19 22:20 _patch_key.json
-rw-r--r-- 1 avidu 197609  790779 Jun 19 22:20 full_f1295.png
-rw-r--r-- 1 avidu 197609  820985 Jun 19 22:20 full_f1330.png
-rw-r--r-- 1 avidu 197609  804891 Jun 19 22:20 full_f1372.png
-rw-r--r-- 1 avidu 197609  815522 Jun 19 22:20 full_f1410.png
-rw-r--r-- 1 avidu 197609  815037 Jun 19 22:20 full_f1450.png
-rw-r--r-- 1 avidu 197609  814468 Jun 19 22:20 full_f1486.png
-rw-r--r-- 1 avidu 197609  821171 Jun 19 22:20 patch_01.png
-rw-r--r-- 1 avidu 197609  391114 Jun 19 22:20 patch_02.png
-rw-r--r-- 1 avidu 197609  893910 Jun 19 22:20 patch_03.png
-rw-r--r-- 1 avidu 197609  401522 Jun 19 22:20 patch_04.png
-rw-r--r-- 1 avidu 197609 1102638 Jun 19 22:20 patch_05.png
-rw-r--r-- 1 avidu 197609  463582 Jun 19 22:20 patch_06.png
-rw-r--r-- 1 avidu 197609  702892 Jun 19 22:20 patch_07.png
-rw-r--r-- 1 avidu 197609  406269 Jun 19 22:20 patch_08.png
-rw-r--r-- 1 avidu 197609  455134 Jun 19 22:20 patch_09.png
-rw-r--r-- 1 avidu 197609  759526 Jun 19 22:20 patch_10.png
-rw-r--r-- 1 avidu 197609  669678 Jun 19 22:20 patch_11.png
-rw-r--r-- 1 avidu 197609 1994937 Jun 19 22:22 tracks_overlay_crop.png
-rw-r--r-- 1 avidu 197609 1202750 Jun 19 22:22 tracks_overlay_full.png
===ANALYSIS SCRIPT===
"""Read-only probe: does WASB's DETECTOR see a 2nd (prominent-court) shuttle during
GX010142's 21-25s 'adjacent-court' rally that the single-target TRACKER discarded?

Grounds the multi-shuttle ('Which shuttle') design question. Pure-Python, no GPU/torch.
Reads the already-cached raw detector candidates (det_raw.jsonl) + the tracked CSV.
"""
import json
import math
import os
from collections import defaultdict, Counter

CACHE = "output/wasb/GX010142_1080p__c4cfa94a792f_wasb_wasbcache/det_raw.jsonl"
CSV = "output/wasb/GX010142_1080p__c4cfa94a792f_wasb.csv"
FPS = 60000 / 1001
W, H = 1920.0, 1080.0
```

--- the owner's anchor window (rally 21-25; current-state cites 21.30-25.09) ---

```
T0, T1 = 21.0, 25.2
F0, F1 = int(T0 * FPS), int(T1 * FPS)
```

crude x-norm buckets (perspective ignored ? first-cut, see doc §1): prominent (central) court vs right-side adjacent court.

```
PROM_LO, PROM_HI = 0.28, 0.74
ADJ_LO = 0.78


def round_xy(x, y, q=8.0):
    return (round(x / q) * q, round(y / q) * q)


def load_det_by_fid(path):
    """frame_id -> set of unique (x,y,score) candidates (dedup the 3x window repetition)."""
    det = defaultdict(dict)  # fid -> {rounded_xy: (x,y,score)}
    with open(path) as f:
        for line in f:
            line = line.strip()
            if not line:
```

```
                continue
            obj = json.loads(line)
            for fid, cands in obj["f"]:
                for c in cands:
                    x, y, score, scale = c
                    key = round_xy(x, y)
                    # keep the highest-score instance of a near-duplicate blob
                    prev = det[fid].get(key)
                    if prev is None or score > prev[2]:
                        det[fid][key] = (x, y, score)
    return {fid: list(d.values()) for fid, d in det.items()}


def load_tracked(path):
    tr = {}
    with open(path) as f:
        next(f)
        for line in f:
            fr, vis, x, y = line.strip().split(",")
            fr = int(fr)
            vis = int(vis)
            try:
                x = float(x); y = float(y)
            except ValueError:
                x = y = float("-inf")
            tr[fr] = (vis, x, y)
    return tr


def main():
    if not os.path.exists(CACHE):
        raise SystemExit(f"missing {CACHE}")
    det = load_det_by_fid(CACHE)
    tr = load_tracked(CSV)

    # ---------- GLOBAL: how often does the detector see >1 shuttle candidate? ----------
    mult = Counter()
    total_cand_frames = 0
    for fid, cands in det.items():
        if cands:
            total_cand_frames += 1
            mult[min(len(cands), 5)] += 1
    print("=== GLOBAL detector-candidate multiplicity (unique blobs/frame, dedup@8px) ===")
    print(f"frames with >=1 candidate: {total_cand_frames} of {len(det)} cached frames")
    for k in sorted(mult):
        label = f"{k}+" if k == 5 else str(k)
        print(f"  {label} candidate(s): {mult[k]} frames
({100*mult[k]/total_cand_frames:.1f}%)")
    multi = sum(v for k, v in mult.items() if k >= 2)
    print(f"  >=2 candidates (multi-shuttle frames): {multi} ({100*multi/total_cand_frames:.1f}%
of detected frames)")

    # ---------- GHOST: most frequent fixed-pixel candidate across the whole clip ----------
    pix = Counter()
    for fid, cands in det.items():
        for (x, y, s) in cands:
            pix[round_xy(x, y, q=4.0)] += 1
    print("\n=== GHOST scan: top fixed-pixel candidate locations (dedup@4px) ===")
    for (px, py), n in pix.most_common(6):
        print(f"  ({px:.0f},{py:.0f})  x-norm={px/W:.3f}  appears in {n} frames")

    # ---------- THE WINDOW: prominent vs adjacent candidates during 21-25s ----------
    print(f"\n=== WINDOW {T0:.1f}-{T1:.1f}s  (frames {F0}-{F1}) ===")
    frames = range(F0, F1 + 1)
    n_frames = 0
```

```
    n_adj = n_prom = n_both = 0
    tracker_xnorms = []
    prom_xnorms = []
    adj_xnorms = []
    sample = []
    for fid in frames:
        cands = det.get(fid, [])
        if not cands:
            continue
        n_frames += 1
        xs = [c[0] / W for c in cands]
        has_adj = any(xn >= ADJ_LO for xn in xs)
        has_prom = any(PROM_LO <= xn <= PROM_HI for xn in xs)
        n_adj += has_adj
        n_prom += has_prom
        n_both += (has_adj and has_prom)
        prom_xnorms += [xn for xn in xs if PROM_LO <= xn <= PROM_HI]
        adj_xnorms += [xn for xn in xs if xn >= ADJ_LO]
        v, tx, ty = tr.get(fid, (0, float("-inf"), float("-inf")))
        if v and math.isfinite(tx):
            tracker_xnorms.append(tx / W)
        if len(sample) < 12 and len(cands) >= 2:
            sample.append((fid, sorted(round(x / W, 3) for x, y, s in cands)))

    def stats(name, arr):
        if not arr:
            print(f"  {name}: (none)")
            return
        arr = sorted(arr)
        mean = sum(arr) / len(arr)
        print(f"  {name}: n={len(arr)} mean x-norm={mean:.3f} "
              f"min={arr[0]:.3f} median={arr[len(arr)//2]:.3f} max={arr[-1]:.3f}")

    print(f"frames with >=1 candidate in window: {n_frames}")
    print(f"  frames with an ADJACENT-court candidate (x-norm>={ADJ_LO}): {n_adj} "
          f"({100*n_adj/max(1,n_frames):.0f}%)")
    print(f"  frames with a PROMINENT-court candidate (x-norm {PROM_LO}-{PROM_HI}): {n_prom} "
          f"({100*n_prom/max(1,n_frames):.0f}%)")
    print(f"  frames with BOTH simultaneously: {n_both} ({100*n_both/max(1,n_frames):.0f}%)")
    stats("tracker chose (visible)", tracker_xnorms)
    stats("PROMINENT-court candidates", prom_xnorms)
    stats("ADJACENT-court candidates", adj_xnorms)
    print("  sample multi-candidate frames (fid -> sorted x-norms):")
    for fid, xs in sample:
        print(f"    {fid}: {xs}")


if __name__ == "__main__":
    main()
```

21. assistant (2026-06-19T16:59:51.627Z)

This confirms the measurement methodology. Critically, I note the prominent-court detection is bucketed by a crude x-norm band (0.28?0.74), and the adjacent court is x-norm ? 0.78. Now let me check the PR #245 review and the multishuttle linker plan, since there's an active PR. Let me also look at the `_patch_key.json` to understand what the visual confirmation actually showed.

22. user (2026-06-19T16:59:53.423Z)


```
{
  "patch_01.png": {
    "moment": 1295,
    "frames": [
      1294,
      1295,
```

```
      1296
    ],
    "loc": "prom",
    "cx": 1180.5,
    "cy": 461.3,
    "x_norm": 0.615
  },
  "patch_02.png": {
    "moment": 1295,
    "frames": [
      1294,
      1295,
      1296
    ],
    "loc": "adj",
    "cx": 1743.8,
    "cy": 115.3,
    "x_norm": 0.908
  },
  "patch_03.png": {
    "moment": 1330,
    "frames": [
      1329,
      1330,
      1331
    ],
    "loc": "prom",
    "cx": 904.2,
    "cy": 261.6,
    "x_norm": 0.471
  },
  "patch_04.png": {
    "moment": 1330,
    "frames": [
      1329,
      1330,
      1331
    ],
    "loc": "adj",
    "cx": 1672.2,
    "cy": 207.0,
    "x_norm": 0.871
  },
  "patch_05.png": {
    "moment": 1372,
    "frames": [
      1371,
      1372,
      1373
    ],
    "loc": "prom",
    "cx": 873.3,
    "cy": 394.0,
    "x_norm": 0.455
  },
  "patch_06.png": {
    "moment": 1372,
    "frames": [
      1371,
      1372,
      1373
    ],
    "loc": "adj",
    "cx": 1695.2,
    "cy": 414.8,
```

```
    "x_norm": 0.883
  },
  "patch_07.png": {
    "moment": 1410,
    "frames": [
      1409,
      1410,
      1411
    ],
    "loc": "prom",
    "cx": 1284.0,
    "cy": 504.5,
    "x_norm": 0.669
  },
  "patch_08.png": {
    "moment": 1410,
    "frames": [
      1409,
      1410,
      1411
    ],
    "loc": "adj",
    "cx": 1786.0,
    "cy": 171.2,
    "x_norm": 0.93
  },
  "patch_09.png": {
    "moment": 1450,
    "frames": [
      1449,
      1450,
      1451
    ],
    "loc": "adj",
    "cx": 1664.6,
    "cy": 142.6,
    "x_norm": 0.867
  },
  "patch_10.png": {
    "moment": 1486,
    "frames": [
      1485,
      1486,
      1487
    ],
    "loc": "prom",
    "cx": 1038.8,
    "cy": 636.3,
    "x_norm": 0.541
  },
  "patch_11.png": {
    "moment": 1486,
    "frames": [
      1485,
      1486,
      1487
    ],
    "loc": "adj",
    "cx": 1574.7,
    "cy": 215.7,
    "x_norm": 0.82
  }
}===PR245 REVIEW MD===
```


Review ? PR #245: "Proposal for Rally Detection Fixes"

Verdict: don't merge this as the basis for blind implementation, and there's nothing here to "fund." This is a docs-only proposal for ~3 small code changes (two functions in this repo + one log line in the sibling repo). Useful diligence ? but the three items split cleanly into **drop one / re-home one / gate one behind measurement**. Every claim below was reproduced against the actual code, not taken from the write-up.

Good news first: the review clearly *read* the code. The W1/R5 splitter termination-safety praise (§D) is **correct** ? `_midpoint_split_index` returns an index in `[1, len-1]`, so both halves are non-empty and strictly smaller and the worklist provably terminates (`tracknet_runner.py:431`, `:506-513`). The pipeline narrative (§1) is also mostly faithful to the code.

Per-claim assessment

Claim	Verdict	Real bug?	Action
A ? `_spread` zero-spread / "divide-by-zero"	Misleading	? No	**Drop** (at most a one-line comment)
B ? `_trim_inactive_tail` sparse-tail density	Accurate (real, low-sev)	? Minor	**Don't blind-fix** ? route into the rally-END track with a Launch Report
C ? `rally_cvat.py` silent row-skip	Partially accurate	? (observability gap)	Valid, but it's in the **sibling repo** ? land it there
D ? W1/R5 splitter praise	Accurate	n/a	Praise is well-founded

A ? `_spread` (`rally_gate.py:37`). The arithmetic is right (`len==2 ? 0.0`), the impact framing is not. `_spread` is only ever called by `estimate_play_axis` over the *entire* video's visible trajectory (thousands of points), never on small slices (`fusion_hybrid.py:106` caches it per-trajectory). And there is **no division by spread anywhere** ? its only consumer is `deadband = deadband_frac * span` (`rally_gate.py:102`), a multiplication; a zero span just disables the jitter deadband (`abs(v) < 0` is always False), which degrades gracefully. So the plan's "potential divide-by-zero" is unfounded. The function already returns `0.0` for `len < 2` *by design* (line 39), so "make `len==2` non-zero" contradicts the existing guard ? and switching `int()?ceil()` would silently shift the p5/p95 indices, `span`, the deadband, and the kept/dropped-rally decisions on *every* video, with zero measured benefit. Cosmetic non-bug; recommend dropping it. (Aside: the `_spread` that actually feeds the feature schema is a *different* function ? `rally_seq.py:43`, non-truncating `np.percentile` ? so even the "future reuse" worry is already covered there.)

B ? `_trim_inactive_tail` (`tracknet_runner.py:402`). Real and well-spotted: `win_count = i - lo + 1` counts *active* frames in the trailing window, not elapsed time, so a lone fast frame after a dead-time gap shorter than `merge_gap` (2.0s) gives density `1/1 = 1.0` and survives the trim. Reproduced through the public API at the production operating point: a rally that truly ends ~2.0s over-extended to **3.87s**. **But this is not a "harden an edge case" change** ? `_trim_inactive_tail` *is* R4, the **default-ON, measured, tuned rally-END trimmer** (`window_inactivity_end=1.5`, `inactivity_min_density=0.30`; flipped on as a measured milestone ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s, n=13; `models.py:150-155`). It is also precisely the project's **#1 remaining quality gap (rally-END detection)**. The proposed denominator swap with `min_density=0.30` left untouched silently *re-defines* that calibrated threshold: simulated against a normal sparse-tracking rally (5?7.5 active-fps, entirely realistic), it collapses a real 6s rally end by **~6 seconds**. The plan's own open-question #1 (few-frames fallback?) is exactly the unresolved part. ? Fold this into the owner-gated rally-END effort (`RALLY_DETECTION_GAP_CLOSURE.md`), flag-gated, with a **Launch Report (both eval rulers + over-seg guard) + dual-set no-regression**, and `min_density` re-tuned for the new denominator. Cheap, safe interim step: add a sparse-tail regression test that documents the current quirk. "Tests updated and passing" is *not* a sufficient DoD for a boundary-behavior change.

C ? `rally_cvat.py` (sibling repo). Accurate: `load_canonical_csv` does `continue` past malformed rows with no logging, and the `src/` tree has no logger at all. A fully-empty CSV is already hard-gated by the CLI (`sys.exit(1)` + `emit_delivery_report`), so only *partial* undercounts slip through silently ? a genuine but minor observability gap. The `logger.warning` is safe (no test asserts silence). **Two caveats:** (1) it lives in **sports-data-collector**, not this repo, so per our one-PR-per-item rule it can't ride this PR ? land it there; (2) prefer fitting the existing Issue/summary observability over introducing a lone logging channel nothing configures.

Doc / process nits
- The plan skips the non-optional **"Decisions locked"** section and has **no success-metrics / measurement section** ? exactly what a behavior-touching change (B) needs.
- "Extends `RALLY_DETECTION_QUALITY_REPORT.md`" is mis-stated ? that's the paper-checkpoint doc; the relevant tracker is the rally-END / gap-closure work.
- The `src/parsers/cvat/rally_cvat.py` anchor is mis-homed (sibling repo).

Recommendation
Request changes. Don't adopt this plan for blind implementation, and drop the "fund a project" framing ? there is no resourcing ask in the PR; this is a localized ~3-spot cleanup. Concretely: **drop A**, **re-home C** to the collector repo, and **promote B** into the measured rally-END track instead of implementing it as written. The contributor's code-reading is genuine and welcome; the gap is calibration (a non-bug escalated to "divide-by-zero," and a measured, default-ON lever treated as a harmless hardening).

23. assistant (2026-06-19T16:59:59.913Z)

The patch_key.json is critical evidence. Look at the "prominent" candidates the design wants to recover: their cy (y-pixel) values jump around wildly ? moment 1295 cx=1180/cy=461, moment 1330 cx=904/cy=261, moment 1372 cx=873/cy=394, moment 1410 cx=1284/cy=504, moment 1486 cx=1038/cy=636. And the x_norm bucket for "prominent" spans 0.455?0.669, while "adjacent" spans 0.82?0.93. Let me run a quick computation on the actual cached data to test the design's core claim about the "continuous rally arc" and check for the precision failure modes ? specifically whether selecting "any sustained prominent track" would also pick up non-rally tracks.

24. user (2026-06-19T17:00:01.966Z)

```
=== GLOBAL detector-candidate multiplicity (unique blobs/frame, dedup@8px) ===
frames with >=1 candidate: 11750 of 11750 cached frames
  1 candidate(s): 8086 frames (68.8%)
  2 candidate(s): 3000 frames (25.5%)
  3 candidate(s): 526 frames (4.5%)
  4 candidate(s): 115 frames (1.0%)
  5+ candidate(s): 23 frames (0.2%)
  >=2 candidates (multi-shuttle frames): 3664 (31.2% of detected frames)

=== GHOST scan: top fixed-pixel candidate locations (dedup@4px) ===
  (1048,732)  x-norm=0.546  appears in 25 frames
  (1316,596)  x-norm=0.685  appears in 23 frames
  (1800,624)  x-norm=0.938  appears in 23 frames
  (1208,600)  x-norm=0.629  appears in 21 frames
  (1672,660)  x-norm=0.871  appears in 21 frames
  (1796,624)  x-norm=0.935  appears in 19 frames

=== WINDOW 21.0-25.2s  (frames 1258-1510) ===
frames with >=1 candidate in window: 229
  frames with an ADJACENT-court candidate (x-norm>=0.78): 213 (93%)
  frames with a PROMINENT-court candidate (x-norm 0.28-0.74): 189 (83%)
  frames with BOTH simultaneously: 173 (76%)
  tracker chose (visible): n=227 mean x-norm=0.866 min=0.670 median=0.875 max=0.980
  PROMINENT-court candidates: n=239 mean x-norm=0.563 min=0.352 median=0.575 max=0.713
  ADJACENT-court candidates: n=292 mean x-norm=0.885 min=0.820 median=0.881 max=0.983
  sample multi-candidate frames (fid -> sorted x-norms):
    1277: [0.669, 0.672]
    1290: [0.933, 0.982]
    1291: [0.636, 0.927, 0.927, 0.982]
    1292: [0.631, 0.632, 0.921, 0.983]
    1293: [0.625, 0.916]
    1294: [0.619, 0.685, 0.912]
    1295: [0.614, 0.614, 0.615, 0.686, 0.908]
    1296: [0.609, 0.906, 0.907]
    1297: [0.603, 0.9, 0.903, 0.906]
    1298: [0.598, 0.599, 0.899]
    1299: [0.594, 0.897]
```

```
    1300: [0.589, 0.59, 0.894, 0.895]
```

25. assistant (2026-06-19T17:00:06.412Z)

The measurements reproduce exactly. Now the key adversarial test: the design claims selecting "any sustained track in the prominent court" is precision-positive. But the GHOST scan reveals fixed-pixel candidates that appear in many frames (e.g. (1048,732) x-norm=0.546 in 25 frames, (1316,596) in 23 frames ? both inside the "prominent" band). These are stationary detector noise. Let me write a quick probe to test the precision failure modes directly: how many *sustained* tracks exist in the prominent band across the whole clip (not just the rally window), and whether a greedy linker would also stitch together non-rally artifacts.

26. user (2026-06-19T17:00:24.679Z)

```
total prominent-band tracks: 404; SUSTAINED (>=1.5s,>=25pts): 37
clip span (s): 499.4
  t=138.37-142.68s dur=4.30s n=255 vexc=  331px menergy=  7.4px/f
  t=233.23-237.32s dur=4.09s n=241 vexc=  403px menergy=  9.7px/f
  t=324.17-328.13s dur=3.95s n=226 vexc=  300px menergy=  8.9px/f
  t=214.06-217.73s dur=3.67s n=219 vexc=  405px menergy=  8.4px/f
  t=285.18-288.55s dur=3.37s n=198 vexc=  367px menergy=  4.9px/f
  t=359.24-362.26s dur=3.02s n=181 vexc=  377px menergy=  7.4px/f
  t=460.69-463.66s dur=2.97s n=179 vexc=  310px menergy=  8.5px/f
  t=438.25-441.12s dur=2.87s n=164 vexc=  551px menergy= 10.3px/f
  t=217.75-220.60s dur=2.85s n=171 vexc=  379px menergy=  6.6px/f
  t=309.61-312.46s dur=2.85s n=172 vexc=  319px menergy=  5.9px/f
  t=196.26-199.03s dur=2.77s n=163 vexc=  310px menergy=  7.7px/f
  t=415.93-418.62s dur=2.69s n=159 vexc=  130px menergy=  7.4px/f
  t= 80.13- 82.68s dur=2.55s n=142 vexc=  119px menergy=  4.8px/f
  t= 43.88- 46.38s dur=2.50s n=147 vexc=  343px menergy= 13.3px/f
  t= 21.54- 24.01s dur=2.47s n=143 vexc=  373px menergy= 10.6px/f
  t=187.17-189.37s dur=2.20s n=122 vexc=  344px menergy=  8.7px/f
  t= 55.11- 57.09s dur=1.99s n=120 vexc=  206px menergy=  8.1px/f
  t=116.25-118.18s dur=1.94s n=116 vexc=  151px menergy=  7.6px/f
  t=199.15-201.08s dur=1.94s n=108 vexc=  194px menergy=  9.4px/f
  t=128.36-130.26s dur=1.90s n=115 vexc=  334px menergy= 11.1px/f
```

27. assistant (2026-06-19T17:00:30.647Z)

This is exactly the adversarial evidence I need. The whole-clip probe shows 37 sustained prominent-band tracks, and the design's selection criteria (sustained, motion energy, vertical excursion) do not distinguish the owner's true rally (t=21.54-24.01, vexc=373, menergy=10.6) from 36 other tracks with near-identical signatures. Let me verify one more thing ? whether the GX010142 golden labels exist to ground "recall-positive", and check the ablation/experiment harness assumptions about how a SET of tracks feeds the windower (the double-count risk).

28. user (2026-06-19T17:00:35.556Z)

```
---manifest entries for 142---
---is GX010142 in any golden set?---
```

29. assistant (2026-06-19T17:00:40.085Z)

Confirmed: GX010142 ? the entire anchor for this design ? has **no golden labels** and is **not in the LOVO manifest**. So the "recovers a G1 miss ? recall-positive" claim cannot currently be measured at all on the anchor clip. This is a decisive finding. Let me do one final check: confirm how many of those 37 sustained prominent-band tracks fall outside the actual rallies by checking the existing WASB CSV rally count for this clip, to quantify the manufactured-FP risk.

30. user (2026-06-19T17:00:45.624Z)

```
csv: ['output/wasb\\GX010142_1080p__c4cfa94a792f_wasb.csv']
served: 44 windows, total 190.3s over a 499s clip
proposal: 62 windows, total 253.0s over a 499s clip
```

31. assistant (2026-06-19T17:00:56.453Z)

I now have everything I need. The current single-track served pipeline already produces 44 windows on this clip ? and the design would add a *parallel* set of up to 37 sustained prominent-band tracks. The double-counting and FP-manufacture risk is concrete and measured. Let me compose the structured adversarial review.

Key measured findings grounding my review:
1. **37 sustained prominent-band tracks** exist across the 499s clip with motion/excursion signatures indistinguishable from the owner's one true rally (t=21.54-24.01: vexc=373px, menergy=10.6 ? sits *mid-pack* among the 37). The "select any sustained prominent track" rule has no discriminator that isolates the real rally.
2. **GX010142 has no golden labels and is not in the LOVO manifest** ? so "recall-positive" is presently unmeasurable on the very anchor clip; the claim is asserted, not measured.
3. The **GHOST scan** shows fixed-pixel candidates *inside* the prominent band (1048,732 / 1316,596) ? stationary artifacts that a sloppy linker stitches into low-but-nonzero-energy tracks.
4. The existing `court_gate` is **subtractive and window-level** (drops majority-out-of-court windows); the proposed Layer-1+Layer-2 is **additive** (surfaces new tracks the single tracker discarded), which is precisely where FPs get manufactured.

32. user (2026-06-19T17:02:03.338Z)

Output does not match required schema: /risks/0: must have required property 'kills_design', /risks/1: must have required property 'kills_design', /risks/2: must have required property 'kills_design', /risks/3: must have required property 'kills_design', /risks/4: must have required property 'kills_design', /risks/5: must have required property 'kills_design'

33. assistant (2026-06-19T17:02:06.596Z)

Each risk needs the `kills_design` field. Let me re-submit with it added to every risk.

34. user (2026-06-19T17:02:48.678Z)

Structured output provided successfully

35. assistant (2026-06-19T17:03:02.936Z)

I have completed the adversarial review. Summary of what I found, grounded in the actual code and cached data:

Verdict: design-needs-revision (no unmitigable flaw, but the "recall-positive AND precision-positive" claim is not supported and is plausibly net-F1-negative as written).

The decisive evidence came from running the design's own Layer-1 linker over the full clip (`output/wasb/GX010142_1080p__c4cfa94a792f_wasb_wasbcache/det_raw.jsonl`):

1. **37 sustained prominent-band tracks across the 499s clip**, and the owner's one true rally (t=21.54?24.01s, vexc=373px, energy=10.6) sits mid-pack ? 11 other tracks are longer with equal/higher excursion. "Select any sustained rally-like prominent track" has no discriminator isolating real rallies from knock-up / between-point / multi-shuttle-drill motion. This is where FPs get manufactured.

2. **The anchor clip is unmeasurable**: GX010142 is not in `output/human_lovo_manifest.json` and has no `*_golden_features.json`, so "recall-positive" is asserted from one hand-checked arc, with zero labelled negatives to prove net F1 doesn't drop.

3. **The recall framing is likely mislabeled precision**: the current single-track pipeline already emits a window over 21?25s (`rally_seg_eval` ? 44 windows on this clip). The design re-attributes an existing prediction to the right court, not recovers a missing one ? so the benefit overlaps what the already-shipped subtractive `court_gate.py` (#243) does, undercutting the additive-linker justification.

4. **Double-count risk is concrete**: 76% of the rally-window frames carry both a prominent AND

adjacent shuttle; selecting "track(s)" plural while the windower still fires produces 2?3 overlapping predictions per interval, which `tolerance_metrics.match_1to1` (strict 1:1) scores as hard FPs.

Relevant files: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\court_gate.py`, `...\backend\pipeline\detectors\rally_gate.py`, `...\backend\eval\tolerance_metrics.py`, `...\backend\eval\rally_seg_eval.py`, `...\scratch\analyze_multishuttle_142.py`, `...\artifacts\multishuttle_142\_patch_key.json`, `...\docs\PROMINENT_COURT_DETECTION.md`.